

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Công Long - Nguyễn Văn Quý

**Tìm hiểu và xây dựng blockchain
định hướng cho doanh nghiệp**

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CỬ NHÂN TÀI NĂNG

Tp. Hồ Chí Minh, tháng 08/2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Nguyễn Công Long - 22120191
Nguyễn Văn Quý - 22120305

Tìm hiểu và xây dựng blockchain định hướng cho doanh nghiệp

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN CNTT
CHƯƠNG TRÌNH CỬ NHÂN TÀI NĂNG

GIẢNG VIÊN HƯỚNG DẪN
TS. Trần Trung Dũng

Tp. Hồ Chí Minh, tháng 08/2026

Lời cam đoan

Chúng tôi xin cam đoan khóa luận tốt nghiệp “*Tìm hiểu và xây dựng blockchain định hướng cho doanh nghiệp*” là công trình nghiên cứu của chính chúng tôi dưới sự hướng dẫn của TS. Trần Trung Dũng. Các nội dung, số liệu và kết quả trình bày trong khóa luận là trung thực. Toàn bộ mã nguồn của hệ thống do nhóm tự thiết kế và hiện thực; trong đó thuật toán đồng thuận Raft được **tự cài đặt hoàn toàn từ đầu**, không sử dụng thư viện đồng thuận có sẵn. Những phần tham khảo từ tài liệu, thư viện mã nguồn mở và công trình của người khác đều được trích dẫn và liệt kê đầy đủ trong mục Tài liệu tham khảo.

Chúng tôi xin chịu hoàn toàn trách nhiệm về lời cam đoan này.

TP. Hồ Chí Minh, tháng 08 năm 2026

Nhóm tác giả

Nguyễn Công Long

Nguyễn Văn Quý

Lời cảm ơn

Trước hết, chúng tôi xin gửi lời cảm ơn chân thành và sâu sắc nhất đến TS. Trần Trung Dũng — giảng viên hướng dẫn — đã tận tình định hướng, góp ý và đồng hành cùng nhóm trong suốt quá trình thực hiện khóa luận. Những trao đổi về kiến trúc hệ phân tán, thuật toán đồng thuận và phương pháp đánh giá hiệu năng là nền tảng quan trọng để chúng tôi hoàn thành công trình này.

Chúng tôi xin cảm ơn quý Thầy Cô Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên — Đại học Quốc gia TP. Hồ Chí Minh, đặc biệt là Chương trình Cử nhân Tài năng, đã trang bị cho chúng tôi kiến thức nền tảng vững chắc về khoa học máy tính, mạng máy tính, hệ điều hành và mật mã học.

Cuối cùng, chúng tôi xin cảm ơn gia đình và bạn bè đã luôn ủng hộ, động viên trong suốt thời gian học tập và nghiên cứu.

Mặc dù đã nỗ lực hết sức, khóa luận không tránh khỏi những thiếu sót. Chúng tôi rất mong nhận được những ý kiến đóng góp quý báu từ quý Thầy Cô và bạn đọc.

Nhóm tác giả

Table of Contents

List of Figures

List of Tables

Abstract

Khóa luận này trình bày quá trình **tìm hiểu, thiết kế và hiện thực một nền tảng blockchain có cấp phép (permissioned blockchain) định hướng cho doanh nghiệp**, được xây dựng hoàn toàn từ đầu (from-scratch) và lấy cảm hứng từ kiến trúc của Hyperledger Fabric. Khác với phần lớn các đề án sử dụng lại một nền tảng blockchain có sẵn, mục tiêu cốt lõi của chúng tôi là *làm chủ công nghệ lõi*: tự cài đặt từng thành phần để hiểu thấu đáo cách một hệ thống sổ cái phân tán hoạt động, từ tầng mạng ngang hàng, thuật toán đồng thuận, máy ảo hợp đồng thông minh, cho đến tầng lưu trữ bất biến và world state.

Hệ thống được tổ chức theo triết lý tách biệt ba trách nhiệm **Execute–Order–Validate** của Hyperledger Fabric, gồm bốn thành phần độc lập giao tiếp qua mạng ngang hàng libp2p: (1) **Core Service** đóng vai trò cổng vào và tầng tính toán, thực thi hợp đồng thông minh biên dịch sang WebAssembly trong môi trường cô lập **wazero** và điều phối quá trình xác nhận (endorsement); (2) **Ordering Service** — trái tim đồng thuận của hệ thống — *tự cài đặt thuật toán Raft từ đầu* với một biến thể bầu lãnh đạo theo độ ưu tiên (priority-based) thay cho cơ chế timeout ngẫu nhiên; (3) **Committing Peer** kiểm tra tính hợp lệ về mật mã của block và ghi vĩnh viễn vào sổ cái append-only kèm world state UTXO trên LevelDB; và (4) **Blockchain Explorer** là giao diện web React hiển thị dữ liệu theo thời gian thực qua Server-Sent Events. Một cơ sở dữ liệu PostgreSQL đóng vai trò bản sao (mirror) bất đồng bộ phục vụ tra cứu và đo lường, hoàn toàn nằm ngoài luồng đồng thuận.

Khóa luận đặc biệt đi sâu vào tầng đồng thuận: mô hình hóa các trạng thái của node, quy trình bầu lãnh đạo xác định, cơ chế nhân bản log và cắt block theo cơ chế lai (hybrid) giữa kích thước lô và thời gian, quá trình giao block (deliver) theo mô hình đăng-ký–phát-tỏa, và cơ chế đồng bộ (sync) bốn pha dựa trên đồng thuận (**commitIndex**, **commitHash**). Chúng tôi cũng trình bày một loạt tối ưu hiệu năng trên đường nóng (hot path) giúp hệ thống tiệm cận mục tiêu **5.000 giao dịch mỗi giây**, cùng phương pháp đo lường ba lớp (k6, submit, commit) để phân tích throughput, độ trễ end-to-end và hiện tượng backlog dưới tải bão hòa.

Từ khóa: blockchain có cấp phép, Hyperledger Fabric, đồng thuận Raft, hợp đồng thông minh WebAssembly, mạng ngang hàng libp2p, mô hình Execute-Order-Validate, sổ cái phân tán, throughput, độ trễ.

Chapter 1

Giới thiệu

1.1 Bối cảnh và động lực

Trong hơn một thập kỷ qua, kể từ khi Satoshi Nakamoto công bố Bitcoin vào năm 2008 [?], công nghệ chuỗi khối (blockchain) đã phát triển từ một hệ thống tiền điện tử đơn lẻ thành một họ công nghệ nền tảng cho nhiều lĩnh vực: tài chính phi tập trung, truy xuất nguồn gốc hàng hóa, quản lý tài sản số, định danh phi tập trung và hợp đồng thông minh. Bản chất của blockchain — một *sổ cái phân tán* (distributed ledger) bất biến, được nhiều bên không hoàn toàn tin tưởng nhau cùng duy trì — mang lại ba thuộc tính đặc biệt giá trị với doanh nghiệp và nhà nước: tính **minh bạch**, tính **bất biến** và khả năng **kiểm toán** mà không cần một bên trung gian duy nhất nắm giữ quyền lực tuyệt đối.

Sự trưởng thành của công nghệ này đã được phản ánh trong định hướng chính sách ở tầm quốc gia. Nghị quyết số 57-NQ/TW ngày 22/12/2024 của Bộ Chính trị về phát triển khoa học, công nghệ, đổi mới sáng tạo và chuyển đổi số quốc gia đã xác định blockchain là một trong các công nghệ chiến lược cần ưu tiên làm chủ [?]. Trước đó, Quyết định số 1236/QĐ-TTg ngày 30/6/2023 đã phê duyệt *Chiến lược quốc gia về ứng dụng và phát triển công nghệ blockchain đến năm 2030* [?], trong đó đặt ra mục tiêu xây dựng hạ tầng mạng blockchain quốc gia “Make in Vietnam” với năng lực xử lý cao, độ trễ thấp và khả năng mở rộng tới hàng trăm nút xác thực phân tán.

Một trong những yêu cầu kỹ thuật then chốt được nêu ra trong các đề án phát triển hạ tầng blockchain quốc gia là khả năng xử lý **tối thiểu 5.000 giao dịch mỗi giây** với độ trễ dưới một giây, đồng thời đảm bảo an toàn dữ liệu và khả năng mở rộng [?]. Đây không phải là những con số dễ đạt được: chúng đòi hỏi hiểu biết sâu sắc về kiến trúc hệ phân tán, thuật toán đồng thuận, tối ưu hiệu năng trên đường truyền mạng và tầng lưu trữ. Quan trọng hơn, để thực sự *làm chủ* công nghệ — chứ không chỉ vận hành lại một sản phẩm mã nguồn mở của nước ngoài — người kỹ sư cần hiểu được từng lớp của hệ thống hoạt động

ra sao, vì sao lại thiết kế như vậy, và đâu là các điểm đánh đổi (trade-off) cốt lõi.

Chính trong bối cảnh đó, khóa luận này đặt ra một mục tiêu học thuật rõ ràng: *tự xây dựng một nền tảng blockchain định hướng doanh nghiệp từ đầu, để hiểu thấu đáo cơ chế bên trong của một hệ thống sổ cái phân tán có cấp phép*. Chúng tôi không nhằm cạnh tranh với các nền tảng công nghiệp đã được kiểm chứng như Hyperledger Fabric [?], mà coi việc tái hiện các nguyên lý thiết kế của chúng — đặc biệt là việc **tự cài đặt thuật toán đồng thuận Raft** [?] — như một con đường để làm chủ tri thức nền tảng, một bước chuẩn bị cho việc tham gia phát triển hạ tầng blockchain quốc gia trong tương lai.

1.2 Blockchain công khai và blockchain có cấp phép

Không phải mọi blockchain đều giống nhau. Một sự phân loại quan trọng — và là nền tảng cho toàn bộ thiết kế của khóa luận — là sự phân biệt giữa blockchain **công khai** (public/permissionless) và blockchain **có cấp phép** (permissioned).

Blockchain công khai như Bitcoin [?] hay Ethereum [?] cho phép bất kỳ ai cũng có thể tham gia mạng, đọc dữ liệu, gửi giao dịch và tham gia vào quá trình đồng thuận một cách ẩn danh. Vì các bên tham gia không có danh tính rõ ràng và có thể có hành vi xấu, các hệ thống này phải dựa vào những cơ chế đồng thuận tốn kém như Proof-of-Work (bằng chứng công việc) — vốn đòi hỏi một lượng năng lượng tính toán khổng lồ — để chống lại tấn công Sybil và đảm bảo an toàn.

Ngược lại, blockchain có cấp phép — nhóm mà hệ thống trong khóa luận này thuộc về — chỉ cho phép các thành viên *được cấp phép* tham gia. Mỗi node có một danh tính mật mã rõ ràng, thường được biểu diễn qua một cặp khóa công khai/bí mật. Nhờ giả định về danh tính này, hệ thống có thể sử dụng các thuật toán đồng thuận nhẹ hơn nhiều (như Raft hoặc PBFT [?]), đạt throughput cao hơn và độ trễ thấp hơn đáng kể so với các blockchain công khai. Bảng 1 tóm tắt những khác biệt cốt lõi.

Table 1. So sánh blockchain công khai và blockchain có cấp phép

Tiêu chí	Công khai	Có cấp phép (dự án này)
Ví dụ điển hình	Bitcoin, Ethereum	Hyperledger Fabric, hệ thống này
Ai được tham gia	Bất kỳ ai	Chỉ thành viên được cấp phép
Định danh	Ẩn danh / bút danh	Danh tính mật mã rõ ràng
Cơ chế đồng thuận	Tổn kém (PoW, PoS)	Nhẹ (Raft, PBFT)
Throughput	Thấp (vài chục TPS)	Cao (hàng nghìn TPS)
Độ trễ	Cao (phút)	Thấp (giây / mili-giây)
Mô hình tin cậy	Không tin cậy nhau	Tin cậy một phần

Mô hình có cấp phép phù hợp tự nhiên với ngữ cảnh doanh nghiệp và khu vực công: các tổ chức tham gia (ngân hàng, cơ quan quản lý, nhà cung cấp trong chuỗi cung ứng) đều biết danh tính của nhau, chịu sự ràng buộc pháp lý, và cần một hệ thống vừa minh bạch vừa có hiệu năng đủ cao để phục vụ nghiệp vụ thực tế. Đây chính là phân khúc mà Hyperledger Fabric nhắm tới, và cũng là định hướng của khóa luận.

1.3 Phát biểu bài toán

Xây dựng một nền tảng blockchain doanh nghiệp hoàn chỉnh từ đầu đặt ra một loạt thách thức kỹ thuật đan xen nhau, mà nếu chỉ sử dụng lại một nền tảng có sẵn thì người học sẽ không bao giờ phải đối mặt một cách trực tiếp:

1. **Bài toán đồng thuận.** Khi nhiều node cùng nhận giao dịch gần như đồng thời, làm thế nào để tất cả thống nhất về một *thứ tự toàn phần* (total order) duy nhất của các giao dịch? Đây là bài toán đồng thuận kinh điển trong hệ phân tán, với kết quả bất khả thi nổi tiếng FLP [?] cho thấy không thể có thuật toán đồng thuận vừa an toàn vừa luôn kết thúc trong mạng bất đồng bộ có lỗi. Việc tự cài đặt một thuật toán đồng thuận đúng đắn (ở đây là Raft) đòi hỏi xử lý cẩn thận các tình huống bầu cử, mất kết nối, và phân ly mạng (split-brain).
2. **Bài toán thực thi hợp đồng thông minh an toàn.** Hợp đồng thông minh phải chạy một cách *xác định* (deterministic) — cùng đầu vào luôn cho cùng kết quả trên mọi node — và *cô lập* (sandboxed) để không thể phá hoại hệ thống chủ. Làm thế nào để hỗ trợ nhiều ngôn ngữ lập trình mà vẫn đảm bảo hai thuộc tính này?

3. **Bài toán xác thực và toàn vẹn.** Làm thế nào để xác thực “ai gửi giao dịch” mà không cần mật khẩu, và đảm bảo dữ liệu sổ cái không thể bị sửa đổi một cách âm thầm?
4. **Bài toán hiệu năng.** Làm thế nào để đạt throughput hàng nghìn giao dịch mỗi giây trong khi vẫn giữ độ trễ ở mức chấp nhận được? Đây là các nút nghẽn cổ chai (bottleneck) trên đường truyền, và làm thế nào để nhận diện cũng như loại bỏ chúng?
5. **Bài toán giao tiếp mạng.** Làm thế nào để các thành phần phân tán giao tiếp một cách an toàn, có định danh mật mã, và chịu được việc node tham gia/rời mạng động?

Mỗi thách thức trên tương ứng với một quyết định thiết kế cụ thể trong hệ thống. Khóa luận này đặt mục tiêu *trình bày và lý giải* từng quyết định đó, dựa trên mã nguồn thực tế đã được hiện thực và kiểm thử.

1.4 Mục tiêu và phạm vi

1.4.1 Mục tiêu

Mục tiêu tổng quát của khóa luận là **thiết kế, hiện thực và đánh giá một nền tảng blockchain có cấp phép định hướng doanh nghiệp, tự xây dựng từ đầu**. Mục tiêu này được cụ thể hóa thành các mục tiêu thành phần:

- Tự cài đặt thuật toán đồng thuận **Raft** từ đầu (không dùng thư viện), bao gồm bầu lãnh đạo, nhân bản log, cắt block và đồng bộ — với một biến thể bầu lãnh đạo theo độ ưu tiên.
- Xây dựng một máy ảo hợp đồng thông minh dựa trên **WebAssembly**, cho phép viết hợp đồng bằng ngôn ngữ bậc cao và thực thi an toàn, xác định.
- Hiện thực mô hình xác nhận giao dịch **Execute–Order–Validate** theo triết lý của Hyperledger Fabric, với chữ ký số Ed25519 và kiểm tra endorsement.
- Xây dựng tầng lưu trữ bất biến (append-only) cho chuỗi block và world state UTXO trên LevelDB với cập nhật nguyên tử.
- Xây dựng giao diện web (Explorer) hiển thị dữ liệu theo thời gian thực.
- Thiết lập phương pháp đo lường hiệu năng và đánh giá hệ thống dưới tải cao, hướng tới mục tiêu 5.000 TPS.

1.4.2 Phạm vi

Để đảm bảo chiều sâu, khóa luận tập trung vào **luồng xử lý cốt lõi** của một giao dịch từ lúc được gửi đến lúc được ghi vĩnh viễn. Cụ thể, báo cáo đi sâu vào bốn thành phần: Core Service, Ordering Service (trọng tâm), Committing Peer và Blockchain Explorer, cùng cơ sở dữ liệu PostgreSQL đóng vai trò mirror.

Một số thành phần phụ trợ phục vụ vận hành — như công cụ điều phối cụm (orchestrator) và giao diện giám sát cụm trong thư mục `orderingservice/` — nằm **ngoài phạm vi** trình bày, vì chúng là công cụ quản trị, không thuộc luồng xử lý giao dịch chính. Tương tự, khóa luận tập trung vào mô hình tin cậy của blockchain có cấp phép (chống lỗi dừng — crash fault tolerance), và chỉ *thảo luận* (chứ không hiện thực đầy đủ) khả năng chịu lỗi Byzantine.

1.5 Đóng góp của khóa luận

Những đóng góp chính của khóa luận bao gồm:

1. **Một hiện thực Raft hoàn chỉnh, tự viết**, với biến thể bầu lãnh đạo theo độ ưu tiên (priority-based leader election) thay cho cơ chế timeout ngẫu nhiên của Raft chuẩn. Biến thể này loại bỏ hiện tượng chia phiếu (split vote) và cho phép hội tụ xác định, đồng thời vẫn giữ nguyên tắc an toàn dựa trên quorum đa số trên tổng số thành viên để chống split-brain.
2. **Một kiến trúc tách tầng Execute–Order–Validate** được hiện thực bằng bốn tiến trình độc lập giao tiếp qua libp2p, minh họa cụ thể cách triết lý thiết kế của Hyperledger Fabric vận hành trong thực tế.
3. **Một máy ảo hợp đồng thông minh WebAssembly** với cơ chế cache module đã biên dịch và pool sandbox dựng sẵn, đạt khả năng thực thi song song hiệu năng cao.
4. **Một loạt tối ưu hiệu năng trên đường nóng** (được đánh số OPT-1 đến OPT-8), giúp tăng số giao dịch trên mỗi block từ khoảng 71 lên hơn 1000 dưới tải bền vững, đưa hệ thống tiệm cận mục tiêu 5.000 TPS.
5. **Một phương pháp đo lường hiệu năng ba lớp** (k6, submit, commit) cho phép phân biệt rõ ràng độ trễ chấp nhận của cổng vào với độ trễ end-to-end, và giải thích hiện tượng backlog dưới tải bão hòa thông qua định luật Little [?].

1.6 Phương pháp tiếp cận

Khóa luận áp dụng phương pháp *xây dựng để hiểu* (build-to-understand): thay vì chỉ phân tích lý thuyết, chúng tôi hiện thực toàn bộ hệ thống bằng ngôn ngữ Go cho ba dịch vụ backend và React cho giao diện, sau đó tiến hành kiểm thử chức năng và đo lường hiệu năng thực nghiệm. Mỗi quyết định thiết kế đều được đối chiếu với (a) nguyên lý lý thuyết tương ứng (bài báo Raft, kiến trúc Fabric, đặc tả WebAssembly...) và (b) kết quả đo lường thực tế. Cách tiếp cận này cho phép phát hiện và lý giải các điểm đánh đổi mà chỉ khi tự hiện thực mới bộc lộ ra, ví dụ như nghẽn cổ chai do tranh chấp khóa khi ghi log theo từng giao dịch, hay sự khác biệt giữa độ trễ HTTP và độ trễ ghi sổ thực sự.

1.7 Kết quả đạt được

Hệ thống đã được hiện thực hoàn chỉnh và chạy được luồng giao dịch end-to-end. Trong các phép đo tải tham khảo (gửi 5.000 yêu cầu mỗi giây trong 60 giây), hệ thống đạt:

- Tốc độ chấp nhận (submit) bền vững khoảng 4.691 giao dịch/giây, đỉnh khoảng 5.017 giao dịch/giây.
- Tốc độ ghi sổ (commit) bền vững khoảng 3.921 giao dịch/giây, đỉnh khoảng 6.000 giao dịch/giây.
- Trung bình khoảng 989 giao dịch trên mỗi block (gần đúng kích thước lô mục tiêu 1.000).

Các kết quả này, cùng với phân tích hiện tượng backlog khi tốc độ gửi vượt tốc độ ghi, được trình bày chi tiết ở chương đánh giá. Chúng cũng là cơ sở để đề xuất các hướng cải thiện trong các chương sau.

1.8 Cấu trúc khóa luận

Phần còn lại của khóa luận được tổ chức như sau:

- **Chương 2 — Các công trình liên quan:** trình bày nền tảng lý thuyết và các công trình tiền đề: Bitcoin và mô hình UTXO, Ethereum và hợp đồng thông minh, Hyperledger Fabric và mô hình Execute–Order–Validate, các

thuật toán đồng thuận (Paxos, Raft, PBFT), WebAssembly cho hợp đồng thông minh, và định vị của khóa luận so với các công trình này.

- **Chương 3 — Phương pháp và Thiết kế hệ thống:** trình bày chi tiết kiến trúc tổng thể, mô hình dữ liệu, và thiết kế của từng thành phần (Core Service, Ordering Service, Committing Peer, Explorer), cơ sở dữ liệu, cùng phương pháp đo lường hiệu năng.
- Các chương tiếp theo (sẽ được hoàn thiện) trình bày hiện thực chi tiết, kết quả thực nghiệm, đánh giá và các đề xuất cải thiện về tốc độ, lưu trữ và bảo mật.

Chapter 2

Các công trình liên quan

Chương này khảo sát nền tảng lý thuyết và các công trình tiền đề mà khóa luận dựa vào hoặc lấy cảm hứng. Chúng tôi đi từ những khái niệm nền tảng nhất (sổ cái phân tán, hàm băm, cây Merkle) đến các hệ thống blockchain tiêu biểu (Bitcoin, Ethereum, Hyperledger Fabric), rồi tới các thuật toán đồng thuận, công nghệ máy ảo hợp đồng thông minh và hạ tầng mạng ngang hàng. Cuối chương, chúng tôi định vị hệ thống của khóa luận trong bức tranh tổng thể đó.

2.1 Sổ cái phân tán và hàm băm mật mã

Cốt lõi của mọi blockchain là một **sổ cái phân tán**: một danh sách các bản ghi (giao dịch) được nhóm thành các *block* (khối), trong đó mỗi block chứa “dấu vân tay” mật mã của block liền trước. Dấu vân tay này được tính bằng một **hàm băm mật mã** (cryptographic hash function) — một hàm biến dữ liệu có độ dài tùy ý thành một chuỗi bit có độ dài cố định, với ba thuộc tính then chốt: kháng tiền ảnh (preimage resistance), kháng tiền ảnh thứ hai, và kháng va chạm (collision resistance). Một thay đổi dù chỉ một bit ở dữ liệu đầu vào cũng làm thay đổi hoàn toàn giá trị băm, và về mặt tính toán là không khả thi để đảo ngược.

Vì mỗi block tham chiếu tới hash của block trước, việc sửa đổi một block cũ sẽ làm thay đổi hash của nó, kéo theo phải tính lại toàn bộ các block sau — điều khiến dữ liệu trở nên gần như **bất biến**. Đây chính là cơ chế “chuỗi” (chain) trong từ “blockchain”. Hệ thống trong khóa luận sử dụng SHA-256 (cụ thể là double-SHA256, giống Bitcoin) [?,?] cho hash block.

2.1.1 Cây Merkle

Để tóm tắt toàn bộ các giao dịch trong một block thành một giá trị duy nhất, blockchain sử dụng **cây Merkle** [?] (Merkle tree, hay hash tree). Trong cấu trúc này, các giao dịch là lá; mỗi nút trong là hash của hai nút con ghép

lại; và nút gốc — *Merkle root* — tóm tắt toàn bộ cây. Cây Merkle mang lại hai lợi ích: (1) bất kỳ thay đổi nào ở một giao dịch đều làm thay đổi Merkle root, nên root đóng vai trò bằng chứng toàn vẹn; và (2) người ta có thể chứng minh một giao dịch thuộc về block mà chỉ cần cung cấp $O(\log n)$ hash trên đường từ lá đến gốc, thay vì tải toàn bộ block — cơ sở cho các “light client” (máy khách nhẹ). Hệ thống của khóa luận tính Merkle root từ danh sách `txid` của các giao dịch và đưa vào header của block.

2.2 Bitcoin và mô hình UTXO

Bitcoin [?] là blockchain đầu tiên giải quyết được bài toán “chi tiêu hai lần” (double-spending) trong môi trường không cần tin cậy, thông qua cơ chế đồng thuận Proof-of-Work và một cấu trúc dữ liệu giao dịch độc đáo: mô hình **UTXO** (Unspent Transaction Output — đầu ra giao dịch chưa tiêu).

Thay vì lưu “số dư tài khoản” như ngân hàng truyền thống, Bitcoin lưu một tập hợp các “tờ tiền” chưa tiêu. Mỗi giao dịch gồm:

- Các **đầu vào** (inputs, ký hiệu `Vin`): tham chiếu và “tiêu” các UTXO đã tồn tại từ trước.
- Các **đầu ra** (outputs, ký hiệu `Vout`): tạo ra các UTXO mới, gán cho người nhận.

Số dư của một địa chỉ chính là tổng giá trị các UTXO mà địa chỉ đó kiểm soát. Mô hình UTXO có ưu điểm về khả năng song song hóa kiểm tra giao dịch và tính riêng tư, nhưng kém trực quan hơn mô hình tài khoản (account model) khi biểu diễn trạng thái phức tạp. Hệ thống của khóa luận **hỗ trợ mô hình UTXO**: world state ở Committing Peer được tổ chức như một tập UTXO, và CLI client đóng vai trò một ví UTXO đầy đủ.

2.3 Ethereum, mô hình tài khoản và hợp đồng thông minh

Ethereum [?,?] mở rộng ý tưởng blockchain từ một hệ thống thanh toán thành một “máy tính thế giới” (world computer) có khả năng lập trình. Đóng

góp lớn nhất của Ethereum là **hợp đồng thông minh** (smart contract): các chương trình được lưu và thực thi trên blockchain, tự động chạy khi có giao dịch gọi tới.

Ethereum sử dụng **mô hình tài khoản** (account-based model) thay cho UTXO: trạng thái toàn cục là một ánh xạ từ địa chỉ tài khoản đến số dư và bộ nhớ hợp đồng. Hợp đồng được biên dịch sang bytecode của *Máy ảo Ethereum* (EVM) và thực thi theo mô hình **Order-Execute**: các giao dịch được sắp xếp trước (qua đồng thuận), sau đó *mọi* node thực thi lại tuần tự để cập nhật trạng thái. Mô hình này đơn giản về mặt khái niệm nhưng hạn chế về hiệu năng và khả năng song song: mọi node phải chạy lại mọi hợp đồng, và một hợp đồng độc hại (ví dụ vòng lặp vô hạn) có thể gây nguy hiểm — vấn đề mà Ethereum giải quyết bằng cơ chế “gas” tính phí cho từng bước tính toán.

Hệ thống của khóa luận hỗ trợ hợp đồng thông minh (qua WebAssembly thay vì EVM) nhưng áp dụng mô hình thực thi khác — Execute-Order-Validate của Fabric — sẽ được phân tích ở phần sau.

2.4 Hyperledger Fabric và mô hình Execute–Order–Validate

Hyperledger Fabric [?] là nền tảng blockchain có cấp phép tiêu biểu cho doanh nghiệp, và là **nguồn cảm hứng kiến trúc chính** của khóa luận. Đóng góp quan trọng nhất của Fabric — so với mô hình Order-Execute của Ethereum — là kiến trúc **Execute–Order–Validate**, tách quá trình xử lý một giao dịch thành ba pha độc lập:

1. **Execute (Thực thi & Xác nhận)**. Giao dịch được gửi tới các *endorsing peer*; mỗi peer thực thi hợp đồng (*chaincode*) trong môi trường cô lập và *ký xác nhận* (endorse) kết quả, nhưng **chưa** ghi vào sổ cái. Vì các peer thực thi độc lập, bước này có thể được song song hóa.
2. **Order (Sắp xếp)**. Các giao dịch đã có chữ ký xác nhận được gửi tới *ordering service*, nơi quyết định thứ tự toàn phần và gom chúng thành block. Đáng chú ý, ordering service **không cần hiểu nội dung** giao dịch — nó chỉ sắp xếp, nên rất nhẹ và dễ mở rộng.

3. **Validate (Kiểm tra & Ghi).** Các *committing peer* nhận block đã sắp xếp, kiểm tra chữ ký xác nhận có thỏa mãn chính sách endorsement không, kiểm tra xung đột phiên bản (đọc-ghi), rồi mới ghi vĩnh viễn.

Mô hình này mang lại ba lợi ích cốt lõi: (a) **khả năng song song** ở pha thực thi; (b) **tính module hóa**, cho phép thay thế cơ chế đồng thuận mà không ảnh hưởng tới logic nghiệp vụ; và (c) **bảo mật theo lớp**, vì mỗi tầng chỉ thấy thông tin nó cần. Fabric thật sử dụng thư viện etcd/raft cho ordering service.

Hệ thống của khóa luận **tái hiện trung thành** triết lý này: Core Service đóng vai trò endorsing peer/gateway (pha Execute), Ordering Service đóng vai trò ordering service (pha Order), và Committing Peer đóng vai trò committing peer (pha Validate). Điểm khác biệt học thuật quan trọng là chúng tôi **tự cài đặt thuật toán Raft từ đầu** thay vì dùng thư viện, và đơn giản hóa một số khía cạnh (ví dụ chính sách endorsement) để tập trung vào việc làm chủ cơ chế lõi. Bảng 2 đối chiếu các vai trò.

Table 2. Đối chiếu vai trò giữa Hyperledger Fabric và hệ thống của khóa luận

Vai trò trong Fabric	Thành phần trong dự án	Pha
Endorsing Peer / Gateway	Core Service	Execute
Ordering Service (etcd/raft)	Ordering Service (Raft tự viết)	Order
Committing Peer	Committing Peer	Validate
Chaincode (Go/Java/Node)	Hợp đồng WASM (TinyGo)	Execute

2.5 Bài toán đồng thuận và các thuật toán tiêu biểu

Đồng thuận (consensus) là bài toán trung tâm của hệ phân tán: làm thế nào để nhiều tiến trình, một số có thể bị lỗi, cùng thống nhất về một giá trị (hoặc một chuỗi giá trị) duy nhất. Kết quả bất khả thi FLP [?] chứng minh rằng trong một hệ bất đồng bộ hoàn toàn, không tồn tại thuật toán đồng thuận *tất định* vừa luôn an toàn vừa luôn kết thúc khi có dù chỉ một tiến trình lỗi. Các thuật toán thực tế “né” kết quả này bằng cách dựa vào giả định thời gian (timeout) để đảm bảo tính kết thúc (liveness) trong khi luôn giữ tính an toàn (safety).

2.5.1 Paxos

Paxos [?] của Leslie Lamport là thuật toán đồng thuận tất định đầu tiên được chứng minh đúng đắn, và là nền tảng lý thuyết cho nhiều hệ thống. Tuy nhiên, Paxos nổi tiếng là *khó hiểu* và khó hiện thực đúng, đặc biệt khi mở rộng thành Multi-Paxos để đồng thuận một chuỗi giá trị.

2.5.2 Raft

Raft [?] được Ongaro và Ousterhout thiết kế với mục tiêu tường minh là *dễ hiểu* (understandability) mà vẫn tương đương Paxos về năng lực. Raft phân rã bài toán đồng thuận thành ba bài toán con tách biệt:

- **Bầu lãnh đạo (Leader Election):** tại mỗi thời điểm, chọn một node làm *Leader*; các node khác là *Follower*. Thời gian được chia thành các *term* (nhiệm kỳ), mỗi term có tối đa một Leader. Raft chuẩn dùng **timeout ngẫu nhiên**: Follower nào hết giờ chờ trước sẽ trở thành *Candidate* và xin phiếu bầu.
- **Nhân bản log (Log Replication):** chỉ Leader nhận lệnh từ client, ghi vào log của mình, rồi sao chép cho Follower. Khi một entry được sao chép tới **đa số** (majority) node, nó được coi là đã *commit*.
- **An toàn (Safety):** một loạt ràng buộc (chỉ bầu cho candidate có log đủ mới, chỉ commit entry của term hiện tại...) đảm bảo không có hai Leader hợp lệ cùng term và log không bị rẽ nhánh.

Khái niệm **quorum đa số** ($\lfloor N/2 \rfloor + 1$ trên tổng N node) là then chốt: vì hai tập đa số bất kỳ luôn giao nhau ở ít nhất một node, không thể có hai quyết định mâu thuẫn được commit. Nhờ vậy Raft chịu được tối đa $\lfloor (N - 1)/2 \rfloor$ node lỗi dừng. Raft là loại đồng thuận **chịu lỗi dừng** (Crash Fault Tolerant — CFT), giả định các node không có hành vi độc hại, chỉ có thể dừng hoặc chậm.

Khóa luận **tự cài đặt Raft** nhưng thay cơ chế bầu cử bằng timeout ngẫu nhiên bằng một biến thể **bầu lãnh đạo theo độ ưu tiên** (xem Chương 3), nhằm loại bỏ hiện tượng chia phiếu và đạt hội tụ xác định.

2.5.3 PBFT và đồng thuận chịu lỗi Byzantine

Trong môi trường mà các node có thể có hành vi *tùy ý* (kể cả độc hại) — bài toán Tướng Byzantine [?] — người ta cần các thuật toán **chịu lỗi Byzantine**

(Byzantine Fault Tolerant — BFT). PBFT [?] của Castro và Liskov là thuật toán BFT thực tế đầu tiên, chịu được tối đa $\lfloor (N - 1)/3 \rfloor$ node độc hại với chi phí truyền thông $O(N^2)$ thông điệp mỗi vòng. So với CFT, BFT an toàn hơn trong môi trường ít tin cậy nhưng đắt hơn nhiều về truyền thông và phức tạp hơn khi hiện thực.

Cachin và Vukolić [?] đã khảo sát một cách hệ thống các giao thức đồng thuận trong bối cảnh blockchain, chỉ ra rằng việc lựa chọn CFT hay BFT phụ thuộc vào mô hình tin cậy của ứng dụng. Với một blockchain có cấp phép trong nội bộ một liên minh doanh nghiệp đã định danh và chịu ràng buộc pháp lý, mô hình CFT (như Raft) thường là đủ và được ưa chuộng vì hiệu năng cao — đây cũng là lựa chọn của Fabric (với Raft) và của khóa luận này.

Table 3. So sánh các thuật toán đồng thuận

Thuật toán	Mô hình lỗi	Số node lỗi chịu được	Ghi chú
Paxos	Crash	$\lfloor (N - 1)/2 \rfloor$	Khó hiểu, nền tảng lý thuyết
Raft	Crash	$\lfloor (N - 1)/2 \rfloor$	Dễ hiểu; dùng trong dự án
PBFT	Byzantine	$\lfloor (N - 1)/3 \rfloor$	Chi phí $O(N^2)$
PoW (Bitcoin)	Byzantine	< 50% sức tính	Tốn năng lượng

2.6 WebAssembly cho hợp đồng thông minh

Việc lựa chọn công nghệ thực thi hợp đồng thông minh ảnh hưởng trực tiếp tới ba yêu cầu: tính xác định, tính cô lập và khả năng đa ngôn ngữ. **WebAssembly** (WASM) [?,?] là một định dạng mã máy ảo dạng stack, ban đầu được thiết kế cho trình duyệt web nhưng nhanh chóng trở thành lựa chọn phổ biến cho các môi trường thực thi phía máy chủ và blockchain. WASM thỏa cả ba yêu cầu: nó chạy trong một bộ nhớ tuyến tính cô lập (sandbox), có ngữ nghĩa thực thi xác định, và có thể được biên dịch từ nhiều ngôn ngữ bậc cao (Rust, Go, C/C++, AssemblyScript...).

Nhiều nền tảng blockchain hiện đại đã chuyển sang WASM: Polkadot dùng WASM cho runtime, CosmWasm đưa hợp đồng WASM vào hệ sinh thái Cosmos,

và Ethereum từng nghiên cứu eWASM như một thay thế cho EVM. Để giao tiếp với hệ điều hành chủ, WASM phía máy chủ thường dùng **WASI** (WebAssembly System Interface) [?] — một tập giao diện hệ thống chuẩn hóa.

Hệ thống của khóa luận biên dịch hợp đồng viết bằng Go sang WASM bằng **TinyGo**, và thực thi bằng **wazero** — một runtime WASM thuần Go không phụ thuộc thư viện C, giúp đơn giản hóa việc build và triển khai. Giữa host và contract có một giao diện hàm (ABI) quy ước để cấp phát bộ nhớ, truyền payload và ghi world state, sẽ được mô tả chi tiết ở Chương 3.

2.7 Mạng ngang hàng và lưu trữ

2.7.1 libp2p

Các thành phần của một blockchain phân tán cần giao tiếp an toàn và linh hoạt. Thay vì dùng HTTP/gRPC thông thường, hệ thống của khóa luận dùng **libp2p** [?] — ngăn xếp mạng module hóa được tách ra từ dự án IPFS [?] và cũng là nền mạng của Ethereum 2.0. libp2p cung cấp: (a) **định danh mật mã** (PeerID suy ra từ khóa công khai); (b) **địa chỉ tự mô tả** (multiaddr) như `/ip4/127.0.0.1/tcp/6000/p2p/12D3Koo...`; (c) **ghép kênh** (stream multiplexing) cho phép nhiều “làn” dữ liệu song song trên một kết nối; và (d) **thương lượng giao thức** qua các Protocol ID. Những đặc tính này đặc biệt phù hợp với giao tiếp consensus, nơi cần gửi nhiều loại thông điệp (heartbeat, đề xuất block, ACK...) một cách hiệu quả.

2.7.2 LevelDB và lưu trữ nhúng

Để lưu world state cần một cơ sở dữ liệu vừa nhanh vừa nhẹ. **LevelDB** [?] là một thư viện lưu trữ key-value nhúng, chạy ngay trong tiến trình (không cần server riêng), dựa trên cấu trúc LSM-tree (log-structured merge-tree) cho phép ghi tuần tự nhanh. Quan trọng nhất với blockchain, LevelDB hỗ trợ **ghi theo lô nguyên tử** (atomic batch): nhiều thao tác ghi/xóa hoặc được áp dụng toàn bộ, hoặc không gì cả — thuộc tính then chốt để cập nhật world state UTXO một cách nhất quán theo từng block.

2.8 Đánh giá hiệu năng blockchain

Việc đánh giá hiệu năng một blockchain có cấp phép đòi hỏi phương pháp luận cẩn thận. BLOCKBENCH [?] là một trong những khung đánh giá đầu tiên cho private blockchain, đề xuất đo throughput và độ trễ ở nhiều lớp khác nhau của hệ thống. Một bài học quan trọng là cần phân biệt rõ *nơi* đo: độ trễ mà client cảm nhận (HTTP) khác với độ trễ ghi sổ thực sự (end-to-end).

Khi tốc độ gửi giao dịch vượt quá tốc độ xử lý bền vững của hệ thống, hàng đợi giao dịch tích lũy thành **backlog** và độ trễ tăng vọt — hiện tượng có thể giải thích bằng **định luật Little** [?]: số lượng công việc trung bình trong hệ thống bằng tốc độ đến nhân với thời gian lưu trú trung bình ($L = \lambda W$). Khóa luận áp dụng cách đo ba lớp (k6 đo HTTP, submit đo lúc chấp nhận, commit đo lúc ghi sổ) và dùng định luật Little để lý giải hành vi backlog ở điểm bão hòa.

2.9 Định vị của khóa luận

Từ khảo sát trên, có thể định vị hệ thống của khóa luận như sau:

- **Về kiến trúc:** tái hiện mô hình Execute–Order–Validate của Hyperledger Fabric [?], nhưng đơn giản hóa để tập trung vào việc làm chủ cơ chế lỗi.
- **Về đồng thuận:** tự cài đặt Raft [?] từ đầu (so với Fabric dùng thư viện etcd/raft), với biến thể bầu lãnh đạo theo độ ưu tiên — một đóng góp mang tính học thuật, đánh đổi tính tổng quát lấy tính đơn giản và hội tụ xác định.
- **Về hợp đồng thông minh:** dùng WebAssembly [?] (giống Polkadot/CosmWasm) thay vì EVM, với runtime thuần Go.
- **Về mô hình giao dịch:** hỗ trợ cả UTXO (giống Bitcoin [?]) lẫn hợp đồng thông minh ghi world state (giống Ethereum [?]).

Khác biệt cốt lõi so với việc sử dụng lại một nền tảng có sẵn nằm ở *mục tiêu*: khóa luận không hướng tới một sản phẩm thương mại sẵn sàng vận hành, mà hướng tới việc **làm chủ tri thức nền tảng** thông qua việc tự hiện thực từng lớp. Chính vì vậy, báo cáo dành phần lớn dung lượng cho việc lý giải các quyết định thiết kế và các điểm đánh đổi, đặc biệt ở tầng đồng thuận — nội dung được trình bày chi tiết trong Chương 3.

Chapter 3

Phương pháp và Thiết kế hệ thống

Chương này trình bày chi tiết phương pháp luận thiết kế và toàn bộ kiến trúc của hệ thống. Chúng tôi bắt đầu từ cái nhìn tổng thể về kiến trúc bốn thành phần và mô hình Execute–Order–Validate, sau đó đặc tả mô hình dữ liệu chung, rồi đi sâu vào thiết kế của từng thành phần theo đúng hành trình của một giao dịch: Core Service (cổng vào và thực thi), Ordering Service (đồng thuận Raft — trọng tâm), Committing Peer (kiểm tra và ghi sổ), và Blockchain Explorer (giao diện). Cuối chương trình bày cơ sở dữ liệu PostgreSQL và phương pháp đo lường hiệu năng.

3.1 Tổng quan kiến trúc

3.1.1 Triết lý thiết kế: tách ba trách nhiệm

Hệ thống được tổ chức theo triết lý **Execute–Order–Validate** của Hyperledger Fabric (đã giới thiệu ở Mục 2.4), trong đó ba trách nhiệm cốt lõi của việc xử lý một giao dịch được tách thành ba tiến trình độc lập:

Table 4. Ba tầng trách nhiệm và ranh giới của chúng

Tầng	Thành phần	Làm gì	Không làm gì
Execute	Core Service	Chạy hợp đồng, xin endorsement	Không quyết định thứ tự
Order	Ordering Service (Raft)	Quyết định thứ tự, gom block	Không chạy hợp đồng, không xét nội dung
Validate	Committing Peer	Kiểm tra & ghi vĩnh viễn	Không sắp xếp lại thứ tự

Việc tách tiến trình mang lại bốn lợi ích thực tiễn: (1) **chịu lỗi** — Ordering Service gồm nhiều node, chết một node vẫn chạy nhờ Raft; (2) **mở rộng độc lập** — cần thêm sức tính hợp đồng thì thêm Core Service, cần ghi nhanh hơn thì thêm Committing Peer; (3) **bảo mật theo lớp** — mỗi tầng chỉ thấy thông tin

nó cần, ví dụ Ordering Service không cần biết nội dung giao dịch; và (4) **phản ánh đúng thực tế doanh nghiệp**, giống mô hình mà ngành tài chính và chuỗi cung ứng đang dùng.

3.1.2 Sơ đồ tổng thể

Hệ thống gồm bốn thành phần chạy độc lập, giao tiếp với nhau qua mạng ngang hàng libp2p, cùng một cơ sở dữ liệu PostgreSQL đóng vai trò bản sao tra cứu. Hình ?? mô tả kiến trúc và các luồng giao tiếp chính.

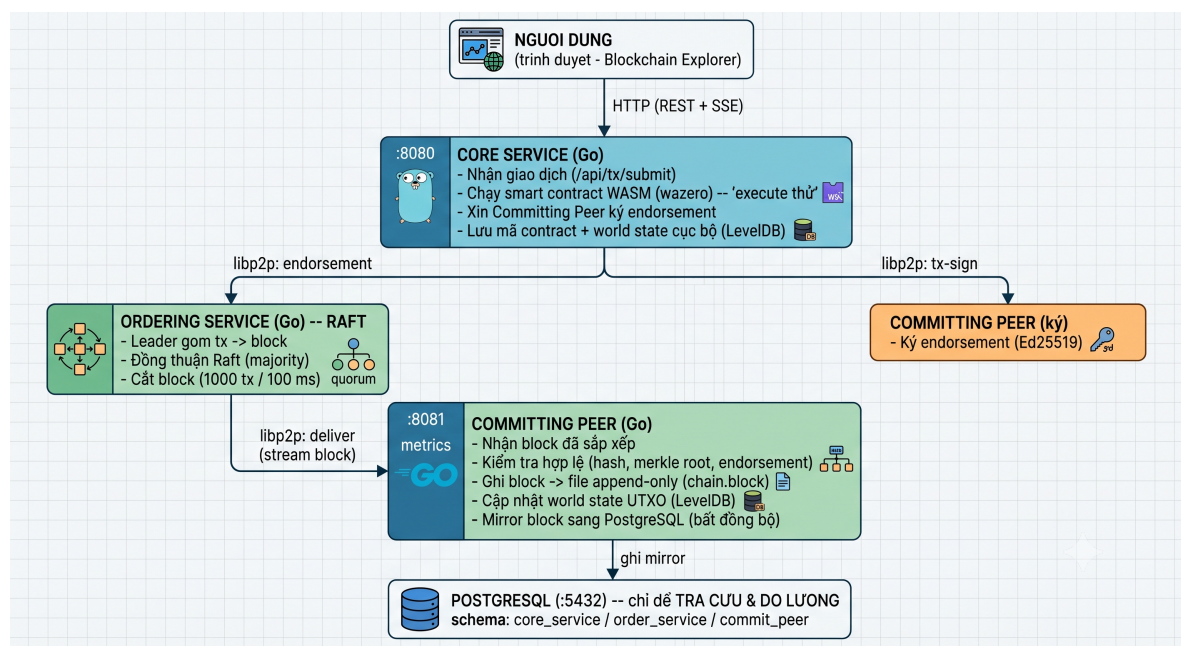


Fig. 1. Kiến trúc tổng thể và các luồng giao tiếp chính của hệ thống.

Một điểm thiết kế quan trọng: PostgreSQL **không** nằm trong luồng đồng thuận. Nó chỉ là bản sao (mirror) để Explorer tra cứu và để đo throughput/latency. Sở cái “thật” nằm ở file `chain.block` và LevelDB của Committing Peer. Nếu PostgreSQL chết, blockchain vẫn chạy bình thường — một quyết định kiến trúc giúp tốc độ ghi sổ không bị phụ thuộc vào tốc độ của một cơ sở dữ liệu ngoài.

3.1.3 Hành trình của một giao dịch

Để hiểu cách các thành phần phối hợp, ta theo dõi hành trình end-to-end của một giao dịch gọi hợp đồng `demo_inventory` để đăng ký 100 sản phẩm:

1. **Tạo & gửi:** Người dùng điền form trên Explorer; frontend mã hóa dữ liệu thành payload nhị phân (hex) và gửi `POST /api/tx/submit` tới Core Service. Giao dịch gồm `txid`, `contract_name`, `function_name`, `payload`.
2. **Execute thử:** Core Service nạp WASM của hợp đồng, chạy hàm kiểm tra với payload. Thành công thì tiếp tục; thất bại trả lỗi ngay.
3. **Xin endorsement:** Core mở stream `tx-sign` tới Committing Peer; peer ký `Ed25519` và trả lại giao dịch kèm chữ ký xác nhận.
4. **Gửi đi sắp xếp:** Core hỏi Discovery ai là Leader của cluster Raft, rồi gửi giao dịch (đã endorse) qua stream `endorsement` tới Leader; đồng thời ghi thời điểm gửi để đo latency.
5. **Đồng thuận & cắt block:** Leader gom giao dịch vào `TxPool`; khi đủ 1000 giao dịch *hoặc* sau 100ms, cắt một block, tính hash + Merkle root, gửi đề xuất cho Follower. Khi đa số Follower ACK, Leader commit block.
6. **Giao block:** Block vừa commit được phát tỏa (fan-out) qua stream `deliver` tới mọi Committing Peer.
7. **Kiểm tra & ghi:** Committing Peer kiểm tra hash, Merkle root, endorsement; nếu hợp lệ thì ghi block vào `chain.block` và cập nhật world state UTXO trong LevelDB (một thao tác ghi nguyên tử).
8. **Mirror & hiển thị:** Committing Peer ghi bản sao sang PostgreSQL bất đồng bộ; Core đọc lại và đẩy cập nhật về trình duyệt qua SSE để Explorer hiển thị gần như tức thì.

3.2 Mô hình dữ liệu

Trước khi đi vào từng thành phần, cần thống nhất các cấu trúc dữ liệu được dùng xuyên suốt hệ thống.

3.2.1 Giao dịch (Transaction)

Hệ thống hỗ trợ **đồng thời hai mô hình** giao dịch: mô hình hợp đồng thông minh (gọi hàm, ghi world state) và mô hình UTXO (Vin/Vout). Cấu trúc `Transaction` (trong `coreservice/internal/core/model.go`) phản ánh cả hai:

```

1 type EndorsementEntry struct {
2     PublicKey string // khoa công khai của peer xác nhận
3     Signature string // chu ký
4 }
5
6 type Transaction struct {
7     Txid string
8     ContractName string
9     FunctionName string
10    Payload []byte // nhị phân, JSON mã hóa hex
11    Endorsements []EndorsementEntry // mảng chu ký xác nhận
12    Signature string // (legacy) phân chieu endorsement cuối
13    SenderPubKey string // (legacy) khoa của endorser cuối
14    Vin []VIN // họ tro UTXO: đầu vào
15    Vout []VOUT // họ tro UTXO: đầu ra
16    Version uint32
17    LockTime uint32
18 }

```

Listing 3.1. Cấu trúc Transaction và Endorsement

Hai trường `Signature` và `SenderPubKey` là kiểu cũ (một chữ ký); hệ thống mới ưu tiên mảng `Endorsements[]` để hỗ trợ *nhiều* chữ ký xác nhận. Khi serialize JSON, `Payload` được mã hóa hex thông qua các hàm `MarshalJSON/UnmarshalJSON` tùy chỉnh, nhằm biểu diễn dữ liệu nhị phân an toàn trong JSON.

3.2.2 Block

Một block tóm tắt một tập giao dịch đã được sắp xếp, với một header kiểu Bitcoin:

- **Timestamp:** thời điểm tạo block.
- **PrevHash:** hash của block trước (nối chuỗi).
- **MerkleRoot:** gốc cây Merkle tính từ danh sách `txid`.
- **Nonce:** giá trị phụ trong header.
- **Hash:** double-SHA256 của header — “dấu vân tay” của block.
- **Transactions:** danh sách giao dịch trong block.

Block hash được tính bằng **double-SHA256** của header (giống Bitcoin), và phải được tính *y hệt* ở cả ba thành phần sinh/kiểm tra block (Ordering Service khi tạo, Committing Peer khi kiểm tra) để đảm bảo nhất quán.

3.2.3 World State và mô hình UTXO

World state là ảnh chụp trạng thái hiện tại của hệ thống, tính được từ toàn bộ lịch sử block. Trong khi chuỗi block là lịch sử bất biến (chỉ thêm), world state là bản đồ key→value có thể ghi đè. Với mô hình UTXO, world state là một *tập các UTXO chưa tiêu*, với lược đồ key:

$$\text{utxo}:\langle \text{txid} \rangle : \langle \text{vout_index} \rangle \longrightarrow \text{JSON của VOUT}$$

Khi áp dụng một block, mỗi VIN sẽ *xóa* một UTXO (đánh dấu đã tiêu), mỗi VOUT sẽ *tạo* một UTXO mới — toàn bộ trong một thao tác ghi nguyên tử (xem Mục ??).

3.2.4 Log entry của Raft

Ở tầng đồng thuận, mỗi đơn vị nhân bản là một LogEntry bao bọc một block:

```
1 type LogEntry struct {
2     Index int64 // vị trí trong log
3     PrevLogIndex int64 // index của entry liên trước (kiểm tra liên tục)
4     Term int64 // term khi tạo
5     Type string // "BLOCK_PROPOSING"
6     Block Block // block thực sự
7 }
```

Listing 3.2. Cấu trúc LogEntry trong Raft

Trường PrevLogIndex cho phép Follower kiểm tra log có **liên tục** hay không (không bị nhảy cóc) trước khi chấp nhận entry mới — đây là cốt lõi đảm bảo an toàn của Raft.

3.3 Core Service — Cổng vào và Tầng tính toán

3.3.1 Vai trò

Core Service (coreservice/, viết bằng Go, cổng HTTP :8080) là cổng vào và đồng thời là tầng tính toán của hệ thống, tương ứng vai trò Endorsing Peer/Gateway trong Fabric. Mọi giao dịch của người dùng đều đi qua đây đầu

tiên. Ba nhiệm vụ chính: (1) thực thi hợp đồng thông minh WASM trong sandbox cô lập để “chạy thử”; (2) điều phối endorsement và gửi giao dịch tới Leader của Ordering Service; (3) phục vụ REST API, SSE realtime và tổng hợp số liệu benchmark.

Core Service gần như *không trạng thái* (stateless) đối với sổ cái: trạng thái chốt cuối nằm ở Committing Peer; Core chỉ giữ bản cache cục bộ (mã contract, world state demo) và đọc lại từ PostgreSQL khi cần hiển thị.

3.3.2 Luồng khởi động

Khi chạy (`cmd/node/main.go`), Core Service thực hiện tuần tự: (1) dựng libp2p host lắng nghe `/ip4/0.0.0.0/tcp/0`; (2) kết nối PostgreSQL với thử lại 10 lần — nếu thất bại vẫn chạy ở chế độ hạn chế; (3) mở hai instance LevelDB (`contract_db` và `ledger_db`), thử lại 3 lần và xóa file LOCK nếu kẹt; (4) tạo `WasmEngine` với pool module; (5) nhập địa chỉ Orderer & Committing Peer rồi làm âm kết nối; (6) đăng ký route HTTP và chạy server; (7) chờ tín hiệu dừng để tắt mượt.

3.3.3 Máy ảo hợp đồng thông minh WASM

Khi Core nhận `POST /api/tx/submit`, bước đầu tiên là **simulate** logic hợp đồng trong `WasmEngine` (`internal/vm/engine.go`). Runtime dùng **wazero** (Go-native, không cần CGO) kèm WASI (`wasi_snapshot_preview1`). Bytecode đọc từ LevelDB (`data/contract_db`, key = `contract_name`). Nếu contract từ chối payload hoặc runtime lỗi, API trả HTTP 400 và dừng — chưa ký endorsement, chưa gửi orderer. Phần dưới mô tả ABI, vòng đời một lần `Execute`, cơ chế cache/pool, quy trình biên dịch TinyGo, và ba contract mẫu.

Kiến trúc runtime. Mỗi Core node giữ **một** `wazero.Runtime` dùng chung cho mọi contract. Lúc khởi tạo (`NewWasmEngine`): (i) instantiate WASI (cho `print`, file descriptor tối thiểu); (ii) đăng ký host module `env` export `PutState`; (iii) tạo map `contractCache` và `pools`. `StateDB` tách hai LevelDB: `contract_db` (bytecode WASM + metadata schema) và `ledger_db` (key/value do `PutState` ghi khi simulate). Đây là state **cục bộ Core**, khác world state UTXO trên Committing Peer.

Giao diện host–contract (ABI). Contract TinyGo và host thống nhất qua export/import WASM. Contract **xuất** (`//export`): `allocate(uint32 size)`

→ `uint32` cấp buffer trong linear memory; host ghi payload vào offset trả về. `verify_tx(ptr, size)` → `uint32` (hoặc `execute`) là entry point, trả 1/0. Contract **nhập** qua `//go:wasmimport env PutState: host` đọc `keyPtr/keySize`, `valPtr/valSize` từ RAM sandbox, gọi `StateDB.PutState`; trả 1 nếu ghi LevelDB OK, 0 nếu đọc memory lỗi hoặc DB lỗi.

Trong contract, payload thường được đọc bằng `unsafe.Slice` trên con trỏ `ptr` (pattern `example_asset`, `bench_ping`). Key/value truyền sang `PutState` cũng là con trỏ offset trong linear memory — host không nhận chuỗi Go trực tiếp. Hình ?? tóm tắt thứ tự gọi.

Fig. 2. Giao diện hàm (ABI) giữa host và contract WASM.

Luồng `Execute(ctx, tx)` từng bước.

Bước A — Nạp và biên dịch. `getOrCompile(tx.ContractName)`: đọc `RLock contractCache`; hit thì trả `CompiledModule` (warm). Miss thì `GetContract` từ `LevelDB`, `CompiledModule`, ghi cache dưới `Lock`. Cold start gồm I/O đĩa + biên dịch WASM; warm chỉ còn phần `instantiate` (rẻ hơn nhiều nhưng vẫn có chi phí nếu không có pool).

Bước B — Lấy sandbox. `acquireModule: poolFor` tạo pool lần đầu gặp contract — vòng lặp $N = \text{WASM_POOL_SIZE}$ lần `InstantiateModule` (mặc định 16, max 32), đẩy vào channel có buffer N . `select` non-blocking: có slot rảnh thì lấy ngay; không thì tạo instance overflow (tên `contractName-<serial>`, serial tăng atomic). Mỗi instance cấu hình `WithStartFunctions("_initialize")` để `TinyGo/WASI` init chạy trước khi gọi `export`.

Bước C — Đưa payload vào sandbox. Nếu `len(tx.Payload)>0`: gọi `allocate(len)`, lấy `ptr` từ kết quả WASM, `Memory.Write` copy byte payload (đã decode từ hex JSON ở tầng API). Thiếu `export allocate` → lỗi “thiếu hàm bắt buộc”. Payload rỗng vẫn gọi entry với `ptr=0`, `size=0` (contract phải tự xử lý).

Bước D — Gọi entry. `FunctionName` từ `tx`, mặc định `execute`. Engine thử danh sách fallback: nếu yêu cầu `execute` mà không có `export` thì thử `verify_tx`, và ngược lại — tương thích contract cũ chỉ `export` một tên. `Call(ctx, ptr, payloadLen)`; WASM trap/panic → lỗi runtime; kết quả số 0 → lỗi nghiệp vụ (“bị Smart Contract từ chối”).

Bước E — Trả sandbox. `defer release(runOK)`: nếu thành công, `runOK=true`, đẩy module lại channel; channel đầy thì `Close` instance (thừa sandbox sau burst). Nếu lỗi, `runOK=false`, luôn `Close` — không recycle instance có thể ở trạng thái hỏng.

Hai lớp tối ưu hiệu năng.

(1) *Compiled module cache* — map in-memory theo tên contract, mutex đọc/ghi. Một bytecode chỉ compile một lần cho đến khi restart Core. Phù hợp khi cùng contract phục vụ hàng nghìn tx (benchmark `bench_ping`).

(2) *Module pool* — channel `chan api.Module` giữ sẵn N instance đã link WASI + host env, tránh `InstantiateModule` lặp trên hot path. Cho phép xử lý song song tối đa N tx đồng thời trên cùng contract (mỗi sandbox một linear memory riêng). Vượt N , engine vẫn phục vụ bằng instance tạm, nhưng latency tăng và có thể đóng bỏ sau khi dùng.

Table 5. Tham số và gỡ lỗi WASM engine

Tham số	Mặc định	Ý nghĩa
WASM_POOL_SIZE	16 (max 32)	Số sandbox giữ sẵn / contract
CORE_LOG	tắt	<code>debug</code> : log cold/warm, WASM std-out/stderr

Biên dịch và triển khai contract. Source nằm trong `coreservice/contracts/<ten>`
Build:

```
cd coreservice/contracts && ./build_wasm.sh
# hoặc tung contract:
tinygo build -o my_contract.wasm -target wasi -no-debug -scheduler=none ./
```

Listing 3.3. Biên dịch WASM (TinyGo)

`-target wasi`: link WASI snapshot mà wazero hỗ trợ. `-target wasm` thuần (browser) kéo `gojs` — host báo `module[gojs] not instantiated`. `-scheduler=none`: không spawn goroutine scheduler TinyGo trong module, giảm overhead và dễ lý giải trong sandbox. `-no-debug`: binary nhỏ hơn. Output `my_contract.wasm` deploy qua `POST /api/tx/deploy` (multipart `contract_name` + file) → lưu `contract_db` + tùy chọn `payload_schema` JSONB trên Postgres.

Payload trên wire. Client (Explorer Transfer, k6) gửi payload là chuỗi hex của JSON UTF-8. Core decode hex → []byte trong `Transaction.Payload`

trước khi vào WASM. Contract dùng `encoding/json` parse struct (ví dụ `{id, color, action}`). `coercePayloadFieldsBySchema` phía frontend ép kiểu số trước khi stringify để `json.Unmarshal` sang `int` không lỗi.

Ba hợp đồng mẫu (Bảng ??).

Table 6. Ba hợp đồng mẫu trong repository

Contract	Payload JSON	Logic chính
example_asset	id, color, action	<code>action=="create":</code> key <code>Asset_<id></code> , value = raw JSON payload, qua <code>PutState</code>
demo_inventory	op, sku, qty	<code>op=="register",</code> <code>qty>=0:</code> key <code>Inv_<sku></code>
bench_ping	v (string)	<code>size<=512, 1<=len(v)<=128;</code> validate only, không <code>PutState</code>

`bench_ping` thiết kế cho benchmark throughput: không gọi host import, không ghi LevelDB — đo trần HTTP + WASM + ký + orderer. `example_asset` minh họa contract có side-effect: mọi trường hợp `action != "create"` hoặc `PutState` trả 0 đều reject (`return 0`). Cả ba export `allocate + verify_tx`; engine resolve tên hàm tự động.

Vị trí trong pipeline và giới hạn. WASM simulate chạy **một lần** tại Core trước endorsement. Committing Peer khi ghi block **không** gọi lại `WasmEngine` — chỉ kiểm tra endorsement, Merkle, hash. Do đó: `PutState` trong simulate cập nhật `ledger_db` Core (hữu ích demo/query cục bộ) nhưng **chưa** đồng bộ tự động sang sổ cái commit peer; state contract toàn mạng cần thiết kế thêm (ví dụ ghi state vào tx read-set/write-set, replay khi commit). UTXO world state thật vẫn do commit peer `ApplyBlock` trên LevelDB riêng. Tách bạch này quan trọng khi đọc kết quả Explorer: mirror Postgres phản ánh tx đã commit, không phải mọi key `PutState` cục bộ trên Core.

Tóm tắt chi phí theo pha. Lần đầu deploy contract mới: compile + khởi tạo pool (N instantiate). Steady-state cùng contract: chỉ `allocate + verify_tx` + (tuỳ) `PutState` trên instance có sẵn — đây là pha engine tối ưu nhằm vào. Đổi `WASM_POOL_SIZE` khi benchmark nhiều client song song: pool nhỏ → nhiều overflow/close; pool lớn (tới 32) → RAM cao hơn nhưng ít instantiate đột biến.

3.3.4 Mật mã và quá trình endorsement

Endorsement là lớp mật mã chứng minh một giao dịch smart-contract đã được “chạy thử” hợp lệ trước khi vào ordering. Trong kiến trúc này, **Core Service không tự ký** trên hot-path POST /api/tx/submit: sau `engine.Execute()` (WASM), Core ủy quyền ký cho **Committing Peer** qua libp2p, nhận tx đã có chữ ký, rồi chuyển tiếp orderer. Thuật toán ký (**Ed25519** hoặc **ML-DSA-44**) được chọn tại **commit peer** (`COMMIT_PEER_KEY_ALGO`); Core giữ nguyên pipeline execute/relay và đảm bảo trường `algorithm` không mất trên JSON wire.

Hai scheme ký và vai trò từng tầng.

Ed25519 [?,?] (mặc định): chữ ký trên đường cong Edwards, deterministic, khóa 32 byte, chữ ký 64 byte (128 ký tự hex trên wire). Phù hợp benchmark throughput — ký và serialize nhanh, response HTTP nhỏ.

ML-DSA-44 (FIPS 204, post-quantum): triển khai trên commit peer qua `cloudflare/circl/sign/mldsa/mldsa44`. Khóa công khai 1312 byte (2624 hex), chữ ký 2420 byte (4840 hex). Core **không** gọi thư viện ML-DSA; chỉ thấy payload JSON phình to sau bước tx-sign và khi relay sang orderer/block. Thời gian `signTxViaCommitPeer()` tăng đáng kể so với Ed25519 — nút thất thường ở ký post-quantum, không phải `engine.Execute()`.

Package `coreservice/internal/crypto` chỉ có helper `SignTransaction/VerifyTransaction` Ed25519 (dev/legacy). Production luôn `signTxViaCommitPeer()` → `Signer` pluggable trên commit peer (`internal/crypto/signer.go`).

Table 7. So sánh endorsement Ed25519 và ML-DSA-44 (góc nhìn Core)

Thuộc tính	Ed25519	ML-DSA-44
Cấu hình	<code>COMMIT_PEER_KEY_ALGO=ed25519</code>	<code>mldsa-44</code> trên peer
Tên wire <code>algorithm</code>	<code>ed25519</code>	<code>mldsa-44</code>
Pubkey hex (điển hình)	64 ký tự	2624 ký tự
Chữ ký hex (điển hình)	128 ký tự	4840 ký tự
Ảnh hưởng Core execute	Không đổi	Không đổi
Ảnh hưởng tx-sign / HTTP	Nhẹ	Nặng (ký + JSON lớn)

Thông điệp ký và cấu trúc `EndorsementEntry`. Message layout thống nhất mọi tầng:

```
TxMessage = txid + contractName + string(payload)
```

(payload là []byte đã decode từ hex JSON). Mỗi endorsement: public_key, signature, algorithm. Struct trong internal/core/model.go; serialize JSON với payload dạng hex. Mảng endorsements[] là format chính; sender_pubkey/signature mirror entry cuối cho client cũ.

Khi commit peer ký, handler tx-sign gán algorithm rõ ràng (ed25519 hoặc mldsa-44), self-verify trước khi trả Core. Core merge vào tx rồi json.Encode gửi orderer; struct Endorsement bên orderer cũng có field algorithm để block deliver tới commit peer không mất metadata. Nếu algorithm thiếu (relay cũ), validator commit peer suy luận từ độ dài hex (InferAlgorithmFromWire) — tránh verify Ed25519 nhầm lên chữ ký ML-DSA.

Khóa tin cậy. Commit peer validate block với TRUSTED_ENDORSER_PUBLIC_KEYS: danh sách algo:pubhex hoặc hex thuần (mặc định hiệu Ed25519). Khi chạy ML-DSA, pubkey phải dạng mldsa-44:<hex> — không dùng pubkey Ed25519 cũ. Core không kiểm tra chữ ký lúc submit; tin commit peer đã verify và chỉ ký khi self-check OK.

Luồng POST /api/tx/submit.

```
1 1. Decode JSON Transaction (payload hex -> []byte)
2 2. engine.Execute() -> WASM simulate
3   | fail -> HTTP 400
4   v OK
5 3. signTxViaCommitPeer() -> libp2p tx-sign
6   | peer: SignTx + EndorsementEntry{algorithm}
7   | fail -> HTTP 502
8   v
9 4. sendEndorsementAsync() -> orderer leader (libp2p)
10 5. SubmitRecorder.Record() -> tx_submit_times (async)
11 6. HTTP 200 + signature preview + endorsement_count
```

Listing 3.4. Luong xu ly submit (Core Service)

Bước 1–2. submittedAt := time.Now() ghi nhận sớm cho metric. Decode body; Execute(ctx, tx) chạy WASM (mục ??). Mọi lỗi contract → HTTP 400, không ký.

Bước 3. signTxViaCommitPeer: parse COMMIT_PEER_P2P (comma-separated multiaddr), thử tuần tự. SignTransactionViaCommitPeer: encode tx JSON → stream /fabric-enterprise/commit-peer/tx-sign/1.0.0 → decode {ok, error, tx}. **Sign pool** (CORE_SIGN_POOL, mặc định bật): giữ TCP/libp2p ấm, mỗi request vẫn mở stream mới (một round-trip), timeout CORE_SIGN_TIMEOUT (15s). Tx trả về đã có endorsements[0] với algorithm khớp scheme peer đang

chạy. Với ML-DSA, response body có thể 8–10 KB/tx chỉ riêng phần chữ ký — ảnh hưởng bandwidth k6/Explorer nhưng không đổi logic API.

Bước 4. sendEndorsementAsync: nếu có orderer discovery, `discovery.SendEndorsement` gửi tx JSON (đã endorse) tới Raft leader qua `/raft-order-service/endorsement/1.0.0`. Mặc định `CORE_ASYNC_ENDORSE=1`: goroutine nền, HTTP 200 ngay sau bước 3. `CORE_ENDORSE_FALLBACK=1`: thử follower nếu leader từ chối. Lỗi gửi orderer **không** đổi HTTP response (đã 200) — tx có thể kẹt nếu async fail; benchmark cần theo dõi log orderer.

Bước 5. SubmitRecorder: channel buffer 65536, worker flush batch 512 dòng / 50 ms vào `core_service.tx_submit_times` (ON CONFLICT DO NOTHING). Tắt bằng `CORE_RECORD_SUBMIT=0`. Queue đầy thì drop — ưu tiên không chặn submit. Dùng join E2E với `committed_at` (mirror hoặc API commit peer :8081).

Execute–Order–Validate trên Core. Core = simulate + delegate sign + forward. Orderer không verify chữ ký endorsement (để commit peer làm). Commit peer khi ghi block gọi `VerifyEndorsement(txMessage, algorithm, sig, pub)` và so `algo:pubhex` với trusted set. Tx gian lận (execute lợi dụng lỗ hổng không có trong thiết kế hiện tại) vẫn cần endorsement hợp lệ từ khóa tin cậy — thiếu hoặc sai algorithm/sig → **block rejected**.

Chuyển đổi scheme khi benchmark. So sánh Ed25519 vs ML-DSA trên cùng Core: (i) restart commit peer với `COMMIT_PEER_KEY_ALGO` và file khóa tương ứng (`endorsement.key` / `endorsement.mldsa44.key`); (ii) cập nhật `TRUSTED_ENDORSER_PUBLIC_KEYS` (iii) không đổi Core/WASM/k6 contract. `engine.Execute()` và `TxMessage` giữ nguyên — chỉ bước 3–4 và kích thước JSON thay đổi. Metric E2E vẫn hợp lệ vì `RecordBlock` commit peer không phụ thuộc algorithm.

Table 8. Biến môi trường endorsement trên Core Service

Biến	Mặc định	Tác dụng
<code>COMMIT_PEER_P2P</code>	(bắt buộc)	Multiaddr peer ký; nhiều addr cách nhau dấu phẩy
<code>CORE_ASYNC_ENDORSE</code>	1	Gửi orderer nền sau khi ký
<code>CORE_ENDORSE_FALLBACK</code>	tắt	1: dial follower khi leader lỗi
<code>CORE_SIGN_POOL</code>	bật	Kết nối libp2p ám tới commit peer
<code>CORE_SIGN_TIMEOUT</code>	15s	Timeout mỗi lần tx-sign
<code>CORE_RECORD_SUBMIT</code>	bật	Ghi <code>tx_submit_times</code> (benchmark E2E)

Biến thuật toán (`COMMIT_PEER_KEY_ALGO`, `TRUSTED_ENDORSER_PUBLIC_KEYS`) cấu hình trên **commit peer**, không trên Core — Core chỉ cần `COMMIT_PEER_P2P` trở đúng peer đang chạy scheme mong muốn.

3.3.5 Mạng libp2p và Discovery

Mọi giao tiếp của Core với orderer và committing peer đều qua **stream** libp2p, phân loại bằng Protocol ID (xem Bảng ??). Hai cơ chế quan trọng cho hiệu năng và độ tin cậy:

Connection pool ấm: mở kết nối mới mỗi lần ký rất tốn (bắt tay TCP + libp2p). Core giữ sẵn một *kết nối ấm* tới Committing Peer (`CORE_SIGN_POOL`, mặc định bật); mỗi lần ký chỉ mở một *stream* mới trên kết nối sẵn có, rẻ hơn nhiều. Có cơ chế thử lại 2 lần và tự kết nối lại nếu hỏng.

Discovery — tìm Leader: Ordering Service có nhiều node và Leader có thể đổi. **Discovery Client** khởi tạo với danh sách bootstrap (vài multiaddr orderer), gọi `Refresh()` để lấy *membership view* (danh sách thành viên + ai là Leader) và cache lại (TTL ~8 giây). Khi phát hiện Leader cũ lỗi, `Invalidate()` xóa cache nóng và tạm dùng bản tốt cuối (`lastGood`) trong cửa sổ failover ~2 phút. Cơ chế cache + failover này giúp Core chịu được việc Leader đổi mà không cần khởi động lại.

3.3.6 Lưu trữ

Core dùng hai loại lưu trữ. **LevelDB** (nhúng cục bộ) mở hai instance: **ContractDB** lưu bytecode WASM và schema UI của contract, **LedgerDB** lưu world state demo. **PostgreSQL** là bản sao chỉ-đọc của sổ cái Committing Peer, phục vụ hiển thị Explorer, lưu danh mục contract và đo benchmark.

Một chi tiết tối ưu đáng chú ý là **SubmitRecorder** (`storage/submit_recorder.go`): để đo E2E latency cần ghi thời điểm submit của từng giao dịch, nhưng gọi DB cho *mỗi* giao dịch ở 5000 TPS sẽ làm sập DB. Giải pháp là **batch INSERT qua channel**: mỗi giao dịch chấp nhận đẩy (`txid`, `submitted_at`) vào một channel đệm lớn (~65k); một goroutine nền gom nhiều bản ghi rồi **INSERT** hàng loạt hoặc xả định kỳ. Đây là mẫu *batching* kinh điển để giảm số lần gọi I/O.

3.3.7 REST API và realtime

Core mở HTTP server tại :8080 với các nhóm endpoint chính trong Bảng ??.

Table 9. Các endpoint chính của Core Service

Route	Method	Mục đích
/api/tx/deploy	POST	Tải lên contract WASM (multipart)
/api/tx/submit	POST	Gửi giao dịch: chạy → ký → sắp xếp
/api/contracts	GET	Liệt kê contract đã deploy
/api/blocks	GET	Liệt kê block đã commit
/api/transactions	GET	Liệt kê giao dịch đã commit
/api/explorer/stream	GET (SSE)	Đẩy block/giao dịch mới realtime
/api/metrics/benchmark	GET	Tổng hợp submit + commit + E2E latency
/api/metrics/throughput	GET	Tốc độ commit theo cửa sổ

Endpoint `/api/explorer/stream` dùng **Server-Sent Events (SSE)** — server đẩy dữ liệu một chiều xuống trình duyệt qua một kết nối HTTP giữ mở. Khi có block/giao dịch mới, Core gửi sự kiện `ledger_update`. SSE được chọn thay WebSocket vì luồng dữ liệu ở đây một chiều (server→client), SSE đơn giản hơn và tự kết nối lại.

3.4 Ordering Service — Đồng thuận Raft

Ordering Service là thành phần trọng tâm của khóa luận cả về mặt kỹ thuật lẫn học thuật. Đây là nơi hệ thống hiện thực một cơ chế **đồng thuận phân tán** theo tinh thần Raft [?] nhưng **tự viết hoàn toàn** bằng Go (không dùng thư viện `etcd/raft` hay `hashicorp/raft`), với một biến thể bầu lãnh đạo *theo độ ưu tiên* thay cho bầu cử theo timeout ngẫu nhiên của Raft nguyên bản. Mục này trình bày lần lượt: (i) vai trò và bài toán tổng sắp thứ tự; (ii) kiến trúc và các thành phần bên trong một node; (iii) mô hình thông điệp và cách định tuyến; (iv) vòng đời trạng thái của một node; (v) quản lý thành viên và độ ưu tiên; (vi) cơ chế quá bán (quorum) và cách nó chống phân ly não (split-brain); (vii) quy trình bầu lãnh đạo chi tiết; (viii) nhân bản log và cắt block — đường nóng quyết định throughput; (ix) giao block cho tầng ghi sổ; (x) đồng bộ (sync) khi một node tụt hậu; (xi) tầng mạng; và (xii) các đánh đổi thiết kế.

3.4.1 Vai trò và bài toán tổng sắp thứ tự

Trong một hệ phân tán, nhiều giao dịch đến gần như đồng thời từ nhiều nguồn khác nhau (nhiều Core Service, nhiều client). Nếu để mỗi bản sao sổ cái tự quyết định thứ tự áp dụng các giao dịch đó, các bản sao sẽ nhanh chóng phân kỳ: cùng một tập giao dịch nhưng áp dụng theo thứ tự khác nhau có thể dẫn tới trạng thái cuối cùng khác nhau (đặc biệt với mô hình UTXO, nơi một đầu vào chỉ được tiêu dùng một lần). Nhiệm vụ của Ordering Service là bảo đảm **mọi node trung thực đều quan sát thấy cùng một chuỗi giao dịch theo đúng một thứ tự duy nhất** — thuộc tính này gọi là *tổng sắp thứ tự* (total order). Đây là điều kiện tiên quyết để world state hội tụ giống hệt nhau tại mọi Committing Peer.

Điểm quan trọng về ranh giới trách nhiệm: Ordering Service **không** thực thi hợp đồng thông minh, **không** kiểm tra nội dung nghiệp vụ của giao dịch, và **không** xác minh chữ ký endorsement. Nó chỉ nhận các giao dịch đã được Core Service “chạy thử” và được Committing Peer ký xác nhận, rồi *gom chúng vào các block theo một thứ tự nhất quán* và phân phát các block đó. Việc kiểm tra hợp lệ mật mã được ủy thác cho Committing Peer (Mục ??). Cách phân tách này giữ cho đường nóng đồng thuận nhẹ và nhanh: chi phí trên mỗi giao dịch tại tầng ordering gần như chỉ là chi phí xếp hàng và nhân bản, không kèm chi phí tính toán hợp đồng hay mật mã.

Sổ cái mà Ordering Service duy trì hoàn toàn nằm trong bộ nhớ (RAM): một chuỗi các block đã commit và một nhật ký (log) các entry chưa commit. Đây là một lựa chọn thiết kế có chủ đích, ưu tiên tốc độ; hệ quả và cách bù đắp (cơ chế sync khi node khởi động lại) được bàn ở Mục ??.

3.4.2 Kiến trúc và các thành phần bên trong một node

Mỗi node của Ordering Service là một tiến trình Go độc lập, được mô hình hóa bằng một cấu trúc trung tâm gọi là **RaftNode**. Toàn bộ trạng thái đồng thuận của node được gói trong cấu trúc này, và mọi luồng xử lý (goroutine) đều thao tác trên đó dưới sự bảo vệ của một số khóa (mutex) được phân tách theo chức năng để giảm tranh chấp. Bảng ?? liệt kê các thành phần chính bên trong một node cùng vai trò của chúng.

Table 10. Các thành phần bên trong một node của Ordering Service

Thành phần	Vai trò
Trạng thái đồng thuận	Trạng thái node (Follower/Leader/...), <i>term</i> hiện tại, ID của Leader đang biết. Được bảo vệ bởi một khóa đọc–ghi chung.
Membership view	Bản đồ toàn bộ thành viên đã biết của cluster (sống và chết), kèm thời điểm tham gia, độ ưu tiên và cờ sống/chết; có một số phiên bản (version) để phát hiện thay đổi.
TxPool	Hồ chứa trong RAM các giao dịch đang chờ đóng block; chỉ Leader dùng. Được bảo vệ bằng khóa riêng để không tranh chấp với luồng đồng thuận.
RaftLog	Nhật ký các entry <i>chưa</i> commit; mỗi entry bao một block cùng chỉ số vị trí (index), chỉ số của entry liền trước và <i>term</i> .
OrderingBlock	Chuỗi các block <i>đã</i> commit — phần sổ cái mà node đã chốt.
Deliver Manager	Quản lý danh sách người đăng ký (các Committing Peer) và phát tỏa mỗi block mới commit tới họ.
Sync coordinator	Điều phối việc “đuổi kịp” khi node thiếu block (mới tham gia hoặc vừa hồi sinh).
Transport	Tầng vận chuyển bọc một host libp2p: mở/nhận stream, gửi và phát tỏa thông điệp.
Các kênh (channel)	Hàng đợi nội bộ nối các luồng: kênh thông điệp chung, kênh ACK block, kênh ACK bầu cử, kênh phản hồi trạng thái sync...

Mô hình đồng thời. Khi khởi động, node chạy song song một số goroutine thường trực: (1) một *bộ xử lý thông điệp* rút thông điệp từ kênh chung và điều phối tới hàm xử lý tương ứng; (2) một *bộ giám sát heartbeat* chạy theo nhịp cố định (mỗi giây) để phát hiện Leader mất tích hoặc để tự gửi heartbeat nếu mình là Leader; và (3) các handler stream do libp2p gọi mỗi khi có kết nối đến (thông điệp đồng thuận, giao dịch từ Core, yêu cầu giao block, yêu cầu sync). Ngoài ra, khi trở thành Leader, node còn chạy thêm một *vòng cắt block tự động*. Các luồng này phối hợp qua kênh và mutex, được thiết kế để đường nóng (nhận giao dịch, cắt block, commit) không bị chặn bởi các luồng chậm.

Phân tách khóa. Một điểm đáng lưu ý về hiệu năng là việc tách các mutex theo trách nhiệm: khóa trạng thái đồng thuận, khóa TxPool, khóa membership, khóa sync và khóa vòng cắt block là độc lập nhau. Nhờ đó, việc Leader thêm giao dịch vào TxPool (rất thường xuyên dưới tải cao) không tranh chấp với việc đọc/ghi trạng thái term hay với luồng bầu cử. Đây là nền tảng để hệ thống duy trì throughput cao mà vẫn an toàn về mặt đồng thời.

3.4.3 Mô hình thông điệp và định tuyến

Các node giao tiếp với nhau bằng một kiểu thông điệp thống nhất, gồm: loại thông điệp, *term* của người gửi, ID người gửi, phần dữ liệu kèm theo (tùy loại) và nhãn thời gian. Bảng ?? liệt kê các loại thông điệp và mục đích của chúng, nhóm theo chức năng.

Table 11. Các loại thông điệp của tầng đồng thuận

Nhóm	Thông điệp	Mục đích
Heartbeat	Heartbeat	Leader báo “tôi còn sống” cho các Follower.
	HeartbeatResponse	Follower báo lại cho một Leader “ôi” (stale) biết term/Leader hiện hành để nó tự thoái vị.
Bầu cử	IAmNewLeader	Node ưu tiên cao nhất tuyên bố “tôi là Leader mới” cho toàn cluster.
	LeaderClaimAck	Follower trả lời CHẤP NHẬN/TỪ CHỐI lời tuyên bố đó.
Thành viên	MembershipUpdate	Yêu cầu gia nhập (từ node mới) hoặc phát tỏa toàn bộ danh sách thành viên (từ Leader).
	MembershipAck	Leader xác nhận đã kết nạp và gửi kèm ảnh chụp thành viên.
	MembershipRequest/Response	Truy vấn/đáp danh sách thành viên (chỉ đọc), dùng khi tìm Leader lúc gia nhập.
Giao dịch	TxRequest/TxResponse	Chuyển tiếp giao dịch tới Leader và xác nhận đã nhận.
Block	BlockProposal	Leader đề xuất một entry log (một block) tới các Follower.
	BlockProposalAck	Follower báo đã chấp nhận (hoặc từ chối) đề xuất.
	BlockCommit	Leader thông báo một block đã đạt quá bán và được commit.
Đồng bộ	SyncStatusRequest/Response	Dò trạng thái commit của các peer để chọn nguồn “đuổi kịp”.

Định tuyến hai tốc độ. Điểm thiết kế đáng chú ý ở tầng định tuyến là không phải mọi thông điệp đều đi qua cùng một hàng đợi. Đa số thông điệp (heartbeat, bầu cử, thành viên, đồng bộ) được đẩy vào một *kênh chung* và xử lý tuần tự bởi bộ xử lý thông điệp. Tuy nhiên, hai loại thông điệp có tần suất cực cao trên đường nóng — *giao dịch đến* và *ACK đề xuất block* — được tách ra

xử lý riêng ngay tại handler stream: giao dịch được nạp thẳng vào TxPool (mỗi stream vốn đã chạy trong goroutine riêng), còn ACK block được đưa vào một kênh chuyên dụng. Lý do là: dưới tải hàng nghìn giao dịch mỗi giây, nếu để mọi thứ dồn vào một hàng đợi chung thì các thông điệp điều phối quan trọng (ACK, commit, heartbeat) sẽ bị kẹt sau một hàng dài giao dịch (hiện tượng nghẽn đầu hàng — head-of-line blocking), làm chậm cả cụm. Việc tách kênh giữ cho tín hiệu đồng thuận luôn thông suốt bất kể lưu lượng giao dịch.

3.4.4 Vòng đời và các trạng thái của node

Mỗi node luôn ở một trong bốn trạng thái. So với Raft nguyên bản (Follower / Candidate / Leader), thiết kế này thay *Candidate* bằng **ClaimingLeader** và bổ sung **Syncing**:

- **Follower**: trạng thái mặc định. Node nghe heartbeat từ Leader, nhận và ACK các đề xuất block, đồng thời chuyển tiếp giao dịch mà nó nhận được tới Leader.
- **ClaimingLeader**: node ưu tiên cao nhất còn sống đang tự tuyên bố làm Leader và chờ thu đủ quá bán số phiếu chấp nhận.
- **Leader**: node đã được chọn; nó gom giao dịch, cắt block, điều phối commit, gửi heartbeat và phục vụ giao block cho tầng ghi sổ.
- **Syncing**: node đang kéo các block/entry còn thiếu từ các peer khác để đuổi kịp trạng thái cluster; trong thời gian này nó tạm hoãn xử lý đề xuất và commit để tránh làm việc trùng.

Hình ?? minh họa các phép chuyển trạng thái. Các phép chuyển chính gồm: Follower → ClaimingLeader khi phát hiện Leader hết hạn heartbeat và chính mình là ứng viên ưu tiên cao nhất; ClaimingLeader → Leader khi thu đủ quá bán phiếu chấp nhận; ClaimingLeader → Follower khi hết thời gian mà không đủ phiếu; Leader → Follower khi nhận được bằng chứng có một Leader ở term cao hơn (qua heartbeat hoặc qua phản hồi heartbeat); và bất kỳ trạng thái nào cũng có thể chuyển sang Syncing khi phát hiện mình thiếu dữ liệu (mới gia nhập, hồi sinh sau mất kết nối, hoặc nhận commit cho một block chưa có trong log).

Fig. 3. Sơ đồ chuyển trạng thái của một node trong Ordering Service.

3.4.5 Quản lý thành viên và độ ưu tiên

Mỗi node duy trì cục bộ một *membership view* — bản đồ toàn bộ thành viên cluster mà nó biết. Với mỗi thành viên, view lưu: ID của peer, thời điểm tham gia, một số **độ ưu tiên** (priority) và cờ sống/chết. Ngoài ra view có một số **phiên bản** (version) tăng dần mỗi khi có thay đổi (kết nạp thành viên, đánh dấu sống/chết), dùng để so sánh xem view của ai mới hơn.

Độ ưu tiên gán theo thứ tự tham gia. Khi một node được kết nạp, nó nhận độ ưu tiên bằng số thành viên hiện có tại thời điểm đó. Nghĩa là node đầu tiên có độ ưu tiên 0 (cao nhất), node thứ hai có độ ưu tiên 1, và cứ thế. **Tham gia càng sớm thì số ưu tiên càng nhỏ, và số càng nhỏ thì mức ưu tiên càng cao.** Ứng viên Leader hợp lệ tại mọi thời điểm chính là *node còn sống có độ ưu tiên nhỏ nhất*. Đây là điểm khác biệt cốt lõi so với Raft nguyên bản và là nền tảng cho cơ chế bầu cử xác định ở Mục ??.

Luồng kết nạp. Chỉ Leader mới có quyền kết nạp thành viên mới. Khi nhận được yêu cầu gia nhập, Leader thêm node vào view của mình, phát tỏa toàn bộ view mới cho mọi thành viên, rồi gửi riêng cho node mới một xác nhận kèm ảnh chụp view. Nhờ Leader là điểm tuần tự hóa duy nhất cho các thay đổi thành viên, mọi node cuối cùng hội tụ về cùng một danh sách với cùng cách gán độ ưu tiên.

Đánh dấu chết. Một thành viên bị đánh dấu chết trong hai tình huống: Leader phát hiện không gửi được thông điệp tới một Follower (khi phát tỏa hoặc gửi heartbeat) thì đánh dấu Follower đó chết và phát tỏa view cập nhật; hoặc một Follower phát hiện Leader hết hạn heartbeat thì đánh dấu Leader chết trước khi khởi động bầu cử. Điểm tinh tế là một thành viên bị đánh dấu chết **không** bị xóa khỏi view — nó vẫn được tính vào tổng số thành viên. Lý do liên quan trực tiếp đến cơ chế quá bán chống split-brain, trình bày ngay dưới đây.

3.4.6 Quá bán tính trên tổng số thành viên

Một quyết định an toàn then chốt của thiết kế là: ngưỡng **quá bán** (quorum) được tính trên **tổng số thành viên đã biết** (cả node sống lẫn node đã bị đánh dấu chết), chứ không phải chỉ trên số node đang sống. Cụ thể, nếu tổng số thành viên là N thì ngưỡng quá bán là

$$\text{quá bán} = \left\lfloor \frac{N}{2} \right\rfloor + 1.$$

Ngưỡng này được áp dụng ở hai nơi quan trọng nhất: khi một node tuyên bố làm Leader (cần thu đủ quá bán phiếu chấp nhận), và khi Leader commit một block (cần đủ quá bán ACK).

Vì sao tính trên tổng chứ không trên số sống? Đây chính là lá chắn nền tảng chống **phân ly não** (split-brain). Giả sử một cluster năm node bị chia đôi do sự cố mạng thành hai phân vùng, một bên ba node và một bên hai node. Nếu quá bán được tính theo số node *sống mà mỗi bên quan sát được*, thì bên nào cũng có thể tự cho mình là “đa số” trong tầm nhìn hạn hẹp của nó và bầu ra một Leader riêng — kết quả là hai Leader cùng commit các block khác nhau, sổ cái phân kỳ không thể hòa giải. Ngược lại, khi quá bán luôn tính trên tổng $N = 5$ (ngưỡng là 3), phân vùng hai node **không bao giờ** đạt nổi ba phiếu, nên không thể bầu Leader cũng không thể commit block; chỉ phân vùng ba node mới tiếp tục hoạt động. Tính chất toán học bảo đảm an toàn ở đây là: *hai tập con quá bán bất kỳ của cùng một tập hợp luôn giao nhau*. Nhờ đó không thể tồn tại đồng thời hai quyết định quá bán mâu thuẫn (hai Leader cùng term, hoặc hai block khác nhau cùng vị trí).

Đánh đổi. Cái giá của lựa chọn an toàn này là tính khả dụng (availability) trong một số kịch bản: nếu quá nửa số node thực sự chết cùng lúc, phần còn lại — dù trung thực và kết nối tốt — cũng không đạt được quá bán và do đó ngừng tiến triển (không bầu được Leader, không commit được block) cho đến khi đủ node hồi sinh. Đây là biểu hiện cụ thể của định lý CAP: khi buộc phải chọn, thiết kế ưu tiên **nhất quán** (consistency) hơn khả dụng, một lựa chọn phù hợp với sổ cái tài chính nơi phân kỳ dữ liệu là điều không thể chấp nhận.

3.4.7 Bầu lãnh đạo theo độ ưu tiên

Đây là đóng góp kỹ thuật đặc trưng nhất của Ordering Service. Thay vì cơ chế bầu cử theo timeout ngẫu nhiên của Raft nguyên bản — nơi bất kỳ Follower nào hết giờ trước cũng có thể tự ứng cử, dẫn tới nguy cơ *chia phiếu* (split vote) và phải bầu lại nhiều vòng — thiết kế này dùng độ ưu tiên đã biết trước để làm cho việc bầu cử trở nên **xác định**. Bảng ?? tóm tắt các khác biệt.

Table 12. So sánh Raft nguyên bản và Raft của dự án

Khía cạnh	Raft nguyên bản	Dự án này
Chọn Leader	Timeout ngẫu nhiên ; ai hết giờ trước thì tự ứng cử	Theo độ ưu tiên : node tham gia sớm hơn = ưu tiên cao hơn = được chọn một cách xác định
Trạng thái	Follower / Candidate / Leader	Follower / ClaimingLeader / Leader / Syncing
Cơ chế phiếu	Mỗi node bầu cho một candidate; có thể chia phiếu	Follower <i>chấp nhận</i> lời tuyên bố “tôi là Leader mới” của node ưu tiên cao nhất
Nội dung log	Lệnh tùy ý	Mỗi entry bao đúng một block
Lưu bền	Ghi đĩa (term, log)	Toàn bộ trong RAM (nhanh, mất khi crash, bù bằng sync)
Thư viện	etcd/raft, hashicorp/raft...	Tự viết hoàn toàn

Phát hiện Leader mất tích Mỗi node chạy một vòng giám sát theo nhịp mỗi giây. Nếu node là Leader, nó chỉ gửi heartbeat khi đã một khoảng thời gian (mặc định 2 giây) không phát đi thông điệp block nào — vì bản thân việc đề xuất/commit block cũng đóng vai trò heartbeat, nên khi cluster đang bận cắt block thì không cần tốn thêm gói heartbeat riêng. Nếu node là Follower, mỗi lần giám sát nó kiểm tra khoảng thời gian kể từ tín hiệu cuối nhận được từ Leader; nếu vượt ngưỡng hết hạn heartbeat (mặc định 5 giây) thì coi như Leader đã mất và khởi động quy trình bầu cử. Bảng ?? liệt kê các hằng số thời gian của tầng đồng thuận.

Table 13. Các hằng số thời gian của tầng đồng thuận

Hằng số	Giá trị	Ý nghĩa
Nhịp gửi heartbeat	2 s	Leader gửi heartbeat nếu suốt 2 s không phát block nào
Ngưỡng hết hạn heartbeat	5 s	Follower quá 5 s không nghe Leader → bầu cử
Nhịp giám sát	1 s	Vòng kiểm tra heartbeat chạy mỗi giây
Cửa sổ chờ phiếu	10 s	Thời gian tối đa thu phiếu chấp nhận khi tuyên bố Leader (gấp đôi ngưỡng hết hạn)
Hạn chờ Leader kỳ vọng	15 s	Follower chờ node ưu tiên cao nhất ra tuyên bố (gấp ba ngưỡng hết hạn)
Chờ ACK mỗi block	5 s	Leader chờ đủ quá bán ACK cho một đề xuất block

Quy trình bầu cử Khi một Follower phát hiện Leader mất, quy trình diễn ra như Hình ?? và gồm các bước sau.

Bước 1 — Xác định ứng viên. Node đánh dấu Leader cũ là chết trong view của mình, rồi tra ra node còn sống có độ ưu tiên nhỏ nhất. Có hai khả năng:

- *Chính mình là ứng viên ưu tiên cao nhất:* node chuyển sang trạng thái ClaimingLeader và tiến hành tuyên bố (Bước 2).
- *Người khác mới là ứng viên:* node **không** tự ứng cử. Thay vào đó nó ghi nhận “Leader kỳ vọng” là node kia và đặt một hạn chờ (mặc định 15 giây). Nó lặng lẽ chờ node kia ra tuyên bố. Nếu quá hạn mà không thấy tuyên bố nào, node coi ứng viên đó cũng đã chết, đánh dấu chết trong view, rồi lặp lại Bước 1 để tìm ứng viên ưu tiên kế tiếp. Cơ chế “chờ theo hạn” này chính là thứ ngăn nhiều node cùng đua nhau ứng cử.

Bước 2 — Tuyên bố làm Leader. Ứng viên tăng *term* của mình lên một đơn vị, chuyển sang ClaimingLeader, rồi phát tỏa thông điệp “tôi là Leader mới” (kèm *term* mới và độ ưu tiên của mình) tới **toàn bộ** thành viên đã biết (cả sống lẫn chết, để phòng trường hợp một node bị đánh dấu chết nhầm vẫn nhận được). Sau đó nó chờ thu phiếu chấp nhận trong cửa sổ tối đa 10 giây.

Bước 3 — Follower đánh giá và bỏ phiếu. Khi một node nhận được lời tuyên bố, nó quyết định chấp nhận hay từ chối dựa trên các điều kiện an toàn:

- Nếu node đang *chờ đúng* người tuyên bố này (đây là “Leader kỳ vọng” của nó), nó chấp nhận ngay, cập nhật Leader và *term* theo lời tuyên bố.
- Nếu node đang là Leader hoặc đang ClaimingLeader, nó **luôn từ chối** lời tuyên bố của node khác (xem phân tích chống split-brain ở Mục ??).
- Trong các trường hợp còn lại, node chỉ chấp nhận khi cả ba điều kiện đồng thời thỏa: (a) người tuyên bố *đúng là* node còn sống có độ ưu tiên cao nhất theo view của nó; (b) *term* trong lời tuyên bố *không nhỏ hơn* *term* hiện tại của nó; và (c) Leader hiện tại (nếu còn) thực sự đã hết hạn heartbeat. Để tránh bỏ phiếu vội, nếu view của node vẫn còn ghi nhận một Leader cũ chưa hết hạn, nó sẽ chờ cho tới khi Leader đó thực sự hết hạn rồi mới đánh giá lại — nhờ vậy nó không từ chối oan một lời tuyên bố hợp lệ chỉ vì đồng hồ của nó lệch pha.

Bước 4 — Kết thúc tuyên bố. Ứng viên đếm số phiếu chấp nhận (tính cả phiếu của chính nó). Nếu đạt **quá bán trên tổng số thành viên**, nó chính thức trở thành Leader: bắt đầu gửi heartbeat và khởi động vòng cắt block. Nếu hết cửa sổ 10 giây mà không đủ phiếu, nó **hoàn tác**: giảm term về giá trị cũ và quay lại trạng thái Follower. Việc hoàn tác term rất quan trọng để một lần tuyên bố thất bại không để lại “rác term” làm nhiễu các vòng bầu cử sau.

Fig. 4. Quy trình bầu lãnh đạo theo độ ưu tiên.

Vì sao bầu theo độ ưu tiên? Vì thứ tự ưu tiên được mọi node biết trước và thống nhất (suy ra từ membership view chung), tại mỗi thời điểm luôn có đúng *một* node “đáng làm Leader nhất”. Các node khác không đua ứng cử mà chỉ chờ node đó ra tuyên bố. Điều này loại bỏ hiện tượng chia phiếu vốn buộc Raft nguyên bản phải bầu lại nhiều vòng, giúp thời gian hội tụ ngắn và hành vi dễ suy luận, dễ gỡ lỗi. Khi ứng viên ưu tiên nhất cũng chết, cơ chế “hạn chờ Leader kỳ vọng” tự động đẩy quyền ứng cử xuống node ưu tiên kế tiếp một cách trật tự. Đánh đổi là thiết kế này thiên về sự đơn giản và tốc độ hội tụ trong các kịch bản vận hành thường gặp, thay vì tổng quát hóa cho mọi mô hình lỗi.

3.4.8 Chống phân ly não (split-brain)

Bảo đảm “không có hai Leader cùng commit các block mâu thuẫn” là yêu cầu an toàn quan trọng nhất của tầng đồng thuận. Thiết kế đạt được điều này bằng **nhiều lớp phòng vệ chồng lên nhau**, chứ không dựa vào một cơ chế duy nhất.

Lớp 1 — Quá bán trên tổng số thành viên. Như đã phân tích ở Mục ??, vì hai tập quá bán bất kỳ luôn giao nhau, không thể có hai node cùng đạt quá bán phiếu trong cùng một term. Trong một phân vùng mạng, chỉ phân vùng chứa quá bán thành viên mới có thể bầu được Leader và commit block; phân vùng thiếu số bị đình trệ hoàn toàn.

Lớp 2 — Leader và ClaimingLeader luôn từ chối lời tuyên bố khác. Một node đang là Leader (hoặc đang trong quá trình tuyên bố) sẽ không bao giờ chấp nhận lời tuyên bố làm Leader của node khác. Điều này ngăn một node đang giữ quyền vô tình “nhường ngôi” rồi lại tiếp tục hành xử như Leader ở

term mới, một tình huống có thể sinh ra hai Leader thoắt qua. Thay vì chấp nhận, node giữ nguyên vai trò và sẽ chỉ *thoái vị một cách thụ động* khi có bằng chứng rõ ràng về một term cao hơn (Lớp 3).

Lớp 3 — Thoái vị theo term. Term đóng vai trò đồng hồ logic toàn cục. Khi một Leader nhận được heartbeat từ một Leader khác với term không nhỏ hơn term của mình, nó lập tức thoái vị về Follower. Ở chiều ngược lại, nếu một Leader “ôi” (đang ở term cũ) gửi heartbeat cho một Follower đã biết term mới hơn, Follower sẽ gửi lại một phản hồi báo cho Leader cũ biết term và Leader hiện hành; nhận được phản hồi này, Leader cũ tự thoái vị và chuyển sang đồng bộ. Nhờ cặp cơ chế này, một Leader lỗi thời (chẳng hạn vừa hồi sinh sau khi bị cô lập) không thể tiếp tục commit: ngay khi tiếp xúc lại với phần cluster đang chạy ở term cao hơn, nó bị buộc thoái vị.

Lớp 4 — Kiểm tra tính liên tục của log khi nhận block. Ngay cả khi một đề xuất block lọt tới Follower, Follower vẫn từ chối nếu người gửi không phải Leader mà nó công nhận, nếu term của đề xuất cũ hơn term hiện tại, hoặc nếu đề xuất không nối tiếp liền mạch với log của nó. Chi tiết ở Mục ??.

Phạm vi bảo đảm. Các lớp trên bảo đảm an toàn trong **đa số kịch bản lỗi thường gặp**: một Leader chết và được thay thế, một Follower chết rồi hồi sinh, mạng chia thành một phân vùng đa số và một phân vùng thiểu số, hoặc một Leader cũ tái xuất sau khi bị cô lập. Điều thiết kế *không* bao trùm là hòa giải hai phân vùng từng cùng tồn tại và cùng ghi dữ liệu độc lập trong thời gian dài — vì quá bán trên tổng số thành viên đã ngăn phân vùng thiểu số commit ngay từ đầu, nên tình huống “hai lịch sử cùng hợp lệ cần trộn lại” về nguyên tắc không phát sinh; phân vùng thiểu số khi tái hợp chỉ việc đồng bộ theo phân vùng đa số. Đây là lý do mục tiêu được phát biểu là “không split-brain trong đa số trường hợp”: thiết kế loại bỏ các con đường dẫn tới hai lịch sử mâu thuẫn trong mô hình lỗi đã chọn (mất node, phân vùng đơn giản), thay vì cố xử lý mọi mô hình lỗi Byzantine hay phân vùng phức tạp.

3.4.9 Nhân bản log và cắt block

Đây là **đường nóng** (hot path) quyết định throughput của toàn hệ thống: từ lúc giao dịch vào TxPool của Leader đến lúc block chứa nó được commit và phát đi.

TxPool và vòng cắt block lai Giao dịch mà Leader nhận (trực tiếp từ Core Service, hoặc do một Follower chuyển tiếp) được gom vào TxPool trong RAM. Một vòng cắt block chạy trên Leader theo cơ chế **lai (hybrid)**, kết hợp kích hoạt theo sự kiện và theo thời gian:

- **Khi hàng chờ đã đầy một lô:** nếu số giao dịch trong TxPool đạt tới kích thước lô mục tiêu (mặc định 1000 giao dịch), Leader cắt block **ngay lập tức**, không chờ thêm. Dưới tải cao và bền vững, TxPool gần như luôn đầy, nên các block được cắt liên tiếp nhau ở nhịp bằng đúng thời gian khứ hồi của một chu trình commit — không còn “thuế thời gian” cố định trên mỗi block.
- **Khi hàng chờ chưa đầy:** Leader chờ tối đa một khoảng (mặc định 100 ms) để gom thêm giao dịch, rồi xả (cắt block) phần đang có. Nếu vẫn rỗng thì lặp lại vòng chờ.
- **Một block trong bay tại một thời điểm:** sau khi đề xuất một block, Leader chờ block đó commit xong (hoặc hết một hạn an toàn) rồi mới đề xuất block kế. Nghĩa là không có nhiều block cùng được đề xuất chồng lấn. Ràng buộc này giữ cho ngữ nghĩa chuỗi hash và cơ chế đồng bộ đơn giản, dễ lập luận đúng đắn.

Đánh đổi kích thước lô. Lô lớn nghĩa là ít block hơn cho cùng một khối lượng giao dịch, nên chi phí đồng thuận cố định (một vòng đề xuất-ACK-commit) được chia đều cho nhiều giao dịch → throughput cao hơn. Nhưng khi tải thấp, phải chờ tới hết khoảng thời gian gom mới cắt block → độ trễ trên mỗi giao dịch tăng. Cơ chế lai cân bằng hai thái cực: tải cao thì cắt ngay theo kích thước (tối ưu throughput), tải thấp thì xả theo thời gian (giới hạn độ trễ).

Cấu trúc block và chuỗi hash Khi cắt block, Leader tạo một block với header kiểu Bitcoin, gồm: nhãn thời gian, hash của block trước (*prev hash*), gốc cây Merkle của danh sách giao dịch, một trường nonce, và hash của chính block. Gốc Merkle được tính từ định danh (txid) của các giao dịch; với block lớn (trên 1000 giao dịch), tầng băm đầu tiên của cây Merkle được **song song hóa** trên nhiều lõi CPU để giảm độ trễ tính toán. Hash của block là **double-SHA256** của phần header (băm SHA-256 hai lần liên tiếp), đúng theo cách Bitcoin làm. Việc dùng prev hash làm cho các block móc xích vào nhau: sửa một block ở giữa sẽ làm sai hash của mọi block sau. Điều cốt yếu là hàm băm và cách tính gốc

Merkle phải **trùng khớp tuyệt đối** giữa nơi tạo block (Ordering Service) và nơi kiểm tra block (Committing Peer), nếu không block hợp lệ sẽ bị từ chối oan.

Đề xuất, chấp nhận và commit Chu trình nhân bản một block diễn ra như Hình ??:

(1) **Đề xuất.** Leader lấy các giao dịch đầu hàng từ TxPool, tạo block với prev hash trỏ tới block commit gần nhất, rồi bọc block vào một *entry log* mang chỉ số vị trí kế tiếp, chỉ số của entry liền trước, và term hiện tại. Entry này được thêm vào RaftLog cục bộ của Leader, rồi phát tỏa cho mọi Follower dưới dạng đề xuất block.

(2) **Follower kiểm tra và ACK.** Mỗi Follower khi nhận đề xuất sẽ kiểm tra theo thứ tự: người gửi có đúng là Leader mà nó công nhận không; term của đề xuất có không nhỏ hơn term của nó không; và quan trọng nhất — **kiểm tra tính liên tục của log**: chỉ số entry liền trước trong đề xuất phải khớp với chỉ số cuối cùng trong log của Follower. Nếu khớp, Follower thêm entry vào log của mình và gửi ACK chấp nhận; nếu lệch (log bị “nhảy cóc”, thiếu entry ở giữa), Follower từ chối kèm lý do — đây là tín hiệu để nó chuyển sang đồng bộ. Ngoài ra, nếu entry liền trước tồn tại nhưng lệch term, Follower cắt bỏ phần log xung đột trước khi xử lý tiếp. Việc kiểm tra liên tục này là cốt lõi bảo đảm mọi bản sao log giống hệt nhau theo cùng thứ tự.

(3) **Commit khi đủ quá bán.** Leader thu ACK vào một kênh chuyên dụng và đếm. Ngay khi số ACK (kể cả của chính Leader) đạt **quá bán trên tổng số thành viên**, Leader commit block: nối block vào chuỗi đã commit, cập nhật con trỏ hash cho block kế, thông báo cho tầng giao block, loại các giao dịch vừa đóng block ra khỏi TxPool, và phát tỏa thông điệp commit cho các Follower. Trường hợp cluster chỉ có một node, chính Leader đã tự đủ quá bán nên commit ngay không cần chờ. Nếu hết hạn chờ ACK (mặc định 5 giây) mà chưa đủ quá bán, block không được commit.

(4) **Follower commit.** Khi nhận thông điệp commit, Follower tra entry tương ứng trong log của mình rồi nối block vào chuỗi đã commit và chuyển tiếp cho tầng giao block. Nếu nó không tìm thấy entry đó trong log (đã bỏ lỡ đề xuất tương ứng), nó khởi động đồng bộ để đuổi kịp thay vì commit mù.

Fig. 5. Đường ống nhân bản và commit một block: đề xuất, chấp nhận theo quá bán, commit.

Cần phân biệt hai cấu trúc lưu block ở tầng ordering: *RaftLog* chứa các entry *chưa* commit (block đã đề xuất nhưng chưa chắc chắn), còn *OrderingBlock* chứa các block *đã* commit (đã chốt). Một entry chỉ di chuyển từ khái niệm “trong log” sang “đã commit” khi đạt quá bán ACK; đây chính là ranh giới giữa dữ liệu tạm và dữ liệu bền của tầng đồng thuận.

3.4.10 Giao block cho tầng ghi sổ (Deliver)

Sau khi commit, block cần tới tay các Committing Peer để được kiểm tra và ghi vĩnh viễn. Việc này do một *trình quản lý giao block* đảm nhiệm theo mô hình **đăng ký và phát tủa** (publish–subscribe). Mỗi Committing Peer mở một kết nối lâu dài tới orderer và gửi một yêu cầu kèm chỉ số “tôi đã có tới block thứ mấy”. Orderer lập tức gửi ngay toàn bộ các block đã commit kể từ chỉ số đó (giúp peer bắt kịp lịch sử), rồi *đăng ký* peer vào danh sách nhận để tiếp tục đẩy các block tương lai ngay khi chúng được commit. Kết nối được giữ mở lâu dài — đây là mô hình *đẩy theo dòng* (streaming push), peer không phải hỏi thăm liên tục.

Mỗi người đăng ký có một hàng đợi đệm riêng (sức chứa 64 block). Điểm thiết kế đáng chú ý về độ bền vững: khi phát tủa một block, nếu hàng đợi của một peer đang đầy (peer đó xử lý chậm), block sẽ bị **bỏ qua không chặn** (non-blocking) đối với peer đó, thay vì làm nghẽn luồng commit của cả cluster. Một peer bị bỏ lỡ block như vậy sẽ tự phát hiện lỗ hổng và bắt kịp bằng cơ chế đồng bộ. Nói cách khác, tốc độ của tầng đồng thuận không bị ghi lại bởi peer chậm nhất.

3.4.11 Đồng bộ khi node tụt hậu (Sync)

Vì sổ cái của tầng ordering nằm trong RAM, một node mới tham gia hoặc vừa hồi sinh sau sự cố sẽ thiếu nhiều block. Ngoài ra, một Follower có thể bỏ lỡ vài block do mạng chập chờn. Cơ chế **đồng bộ** lo việc “đuổi kịp” theo mô hình **kéo** (pull): node tụt hậu chủ động đi lấy dữ liệu còn thiếu. Quy trình đồng bộ được kích hoạt trong nhiều tình huống — lần đầu gia nhập, phát hiện gián đoạn heartbeat kéo dài rồi có lại tín hiệu, một Leader cũ vừa thoái vị, hoặc nhận commit cho một block không có trong log — và diễn ra qua các pha sau.

Pha 1 — Khám phá. Node phát một yêu cầu dò trạng thái kèm chỉ số commit hiện tại của mình tới các peer còn sống, rồi thu các phản hồi trong một

cửa sổ ngắn (mặc định 2 giây). Mỗi phản hồi cho biết chỉ số commit, hash của block commit cuối, chỉ số log cuối, ID của Leader mà peer đó biết, và phiên bản membership.

Pha 2 — Chọn mục tiêu. Node gom các phản hồi theo cặp (*chỉ số commit*, *hash commit*) và chọn nhóm có **hiều phiếu nhất**; nếu hòa thì ưu tiên nhóm có chỉ số commit cao hơn. Việc bỏ phiếu theo cặp (chỉ số, hash) bảo đảm node đồng bộ theo *trạng thái được đa số peer đồng thuận*, chứ không bị một peer đơn lẻ (có thể đang lỗi hoặc giữ nhánh sai) dẫn dắt. Leader nằm trong nhóm thắng được ưu tiên chọn làm nguồn tải chính.

Pha 3 — Tải block song song. Khoảng block còn thiếu được chia thành các *shard* (mỗi shard 64 block). Các shard được tải **đồng thời** từ nhiều nguồn theo kiểu xoay vòng để cân tải; nếu tải một shard từ một nguồn thất bại, node thử nguồn kế tiếp. Cách chia shard song song này rút ngắn đáng kể thời gian đuổi kịp khi cần tải hàng nghìn block.

Pha 4 — Xác minh chuỗi hash. Trước khi cài đặt, node kiểm tra tính toàn vẹn của toàn bộ dãy block tải về: prev hash của mỗi block phải khớp hash của block liền trước; hash lưu trong mỗi block phải khớp với hash tính lại độc lập từ header; và hash của block cuối cùng phải khớp với hash mục tiêu đã chọn ở Pha 2. Nếu bất kỳ kiểm tra nào sai, toàn bộ lần đồng bộ bị hủy và thử lại — node không cài đặt dữ liệu chưa được xác minh.

Pha 5 — Cài đặt. Nếu chuỗi hash hợp lệ, node nối các block vào sổ cái, chuyển tiếp chúng cho tầng giao block, cập nhật con trỏ hash, và nếu cần thì kéo nốt các entry log chưa commit (phần vượt quá chỉ số commit). Cuối cùng node rời trạng thái Syncing về Follower.

Về an toàn của chính cơ chế đồng bộ: một node **đang tự đồng bộ sẽ từ chối phục vụ** yêu cầu đồng bộ của node khác, để không lan truyền dữ liệu mà bản thân chưa xác minh xong. Ngoài ra, mỗi node chỉ chạy một tiến trình đồng bộ tại một thời điểm; nhiều kích hoạt đồng thời được gộp làm một.

3.4.12 Tầng mạng

Toàn bộ giao tiếp của Ordering Service dựa trên mạng ngang hàng libp2p. Một tầng vận chuyển bọc host libp2p và cung cấp các thao tác: gửi thông điệp tới một peer, phát tỏa thông điệp tới một tập peer (bất đồng bộ, kèm khả năng gọi lại khi gửi thất bại để đánh dấu peer chết), và mở stream chuyên dụng cho

từng loại lưu lượng. Mỗi loại lưu lượng dùng một *Protocol ID* riêng để libp2p phân tách và định tuyến tới đúng handler. Bảng ?? liệt kê các Protocol ID dùng trong toàn hệ thống.

Table 14. Các Protocol ID libp2p dùng trong hệ thống

Protocol ID	Mục đích
/raft-order-service/1.0.0	Thông điệp đồng thuận (heartbeat, bầu cử, thành viên)
/raft-order-service/endorsement/1.0.0	Core → Orderer: gửi giao dịch đã endorse
/raft-order-service/deliver/1.0.0	Orderer → Committing Peer: dòng block
/raft-order-service/sync/1.0.0	Đồng bộ block/log giữa các orderer
/fabric-enterprise/commit-peer/tx-sign/1.0.0	Core → Committing Peer: xin chữ ký
/committing-peer/sync/1.0.0	Client → Committing Peer: truy vấn UTXO

Kênh nhận giao dịch bền. Giao dịch từ Core Service đi vào qua một stream *giữ mở lâu dài*: thay vì mở một kết nối mới cho mỗi giao dịch, orderer đọc nhiều giao dịch liên tiếp trên cùng một stream. Ở tải cao, việc mở stream cho từng giao dịch sẽ làm bão hòa cơ chế chấp nhận stream của host và trở thành nút thắt cổ chai; giữ stream bền giúp loại bỏ nút thắt đó. Một node nhận giao dịch mà không phải Leader sẽ chuyển tiếp giao dịch tới Leader; nếu là Leader thì nạp thẳng vào TxPool.

3.4.13 Khởi động và gia nhập cluster

Có hai con đường để một node bước vào cluster. Node *đầu tiên* tự khởi động làm Leader của một cluster một-node (nó đương nhiên đủ quá bán một mình). Một node *gia nhập sau* sẽ kết nối tới một peer mới (bootstrap) đã biết, truy vấn để tìm ra Leader hiện tại, rồi gửi yêu cầu gia nhập trực tiếp tới Leader. Nếu cluster đang trong một vòng bầu cử và tạm thời chưa có Leader, node gia nhập sẽ thử lại vài lần với khoảng lùi ngắn cho đến khi có Leader để liên hệ. Sau khi được kết nạp và nhận ảnh chụp thành viên, nếu phát hiện mình còn thiếu block (sổ cái trống trong khi cluster đã có dữ liệu), node tự động chuyển sang đồng bộ để đuổi kịp trước khi tham gia đầy đủ vào việc đồng thuận.

3.4.14 Các đánh đổi thiết kế

Thiết kế của Ordering Service phản ánh một số đánh đổi có chủ đích, cần được nêu rõ để đánh giá đúng phạm vi áp dụng.

Trạng thái trong RAM, không ghi bền. RaftLog và chuỗi block đã commit đều nằm trong bộ nhớ. Ưu điểm là tốc độ: không có chi phí ghi đĩa trên đường nóng commit. Nhược điểm là khi một node khởi động lại, nó mất toàn bộ trạng thái cục bộ và phải dựng lại bằng cơ chế đồng bộ từ các peer còn sống. Đây là lý do cơ chế đồng bộ (Mục ??) không chỉ là tính năng phụ mà là một phần thiết yếu để cluster chịu được việc node hồi sinh.

Ưu tiên nhất quán hơn khả dụng. Vì quá bán tính trên tổng số thành viên, cluster ngừng tiến triển khi không hội đủ quá bán node sống. Đây là lựa chọn có chủ đích: với sổ cái, đình trệ tạm thời chấp nhận được, còn phân kỳ dữ liệu thì không.

Một block trong bay, không đường ống hóa (pipelining). Việc chỉ cho phép một block được đề xuất tại một thời điểm giữ cho ngữ nghĩa chuỗi hash và đồng bộ đơn giản, dễ lập luận đúng đắn, đổi lại giới hạn mức song song ở tầng nhân bản.

Mô hình lỗi. Các bảo đảm an toàn nhắm tới mô hình lỗi *dừng* (node chết/hồi sinh) và *phân vùng đơn giản* (một đa số, một thiểu số), không nhắm tới lỗi Byzantine ở tầng đồng thuận — việc chống giao dịch/khối gian lận được đặt ở tầng kiểm tra mật mã của Committing Peer. Trong mô hình lỗi đã chọn, các lớp phòng vệ ở Mục ?? loại bỏ khả năng split-brain cho phần lớn các kịch bản vận hành thực tế.

3.5 Committing Peer — Kiểm tra và Ghi sổ

Committing Peer là trạm cuối của hành trình một giao dịch. Nó nhận các block đã được Ordering Service sắp xếp và chốt thứ tự, kiểm tra tính hợp lệ mật mã của chúng, rồi ghi vĩnh viễn vào sổ cái. Đây cũng là nơi *sổ cái thật* của hệ thống nằm: một file chuỗi block bất biến và một cơ sở dữ liệu world state; bản sao trên PostgreSQL chỉ là mirror phục vụ tra cứu. Mục này trình bày trọng tâm vào **thiết kế tổ chức** của thành phần: cách các gói (package) con được

phân tách theo trách nhiệm và ghép nối với nhau, đường ống xử lý bằng kênh (channel), mô hình lưu trữ ba lớp, cùng các ranh giới nhất quán và đánh đổi thiết kế.

3.5.1 Vai trò và nguyên tắc thiết kế

Điểm mấu chốt về ranh giới trách nhiệm: Committing Peer **không** chạy thuật toán đồng thuận và **không** sắp xếp lại thứ tự giao dịch. Nó *tin tưởng* rằng block đến từ Ordering Service đã mang thứ tự cuối cùng; nhiệm vụ của nó gồm hai việc: (i) **kiểm tra** rằng block không bị sửa đổi trên đường truyền và mọi giao dịch hợp đồng đều mang chữ ký endorsement hợp lệ từ khóa tin cậy; và (ii) **ghi** block vào sổ cái đồng thời cập nhật world state một cách nguyên tử. Ngoài ra, peer còn đóng ba vai phụ: làm *endorser* ký xác nhận giao dịch cho Core Service, làm *nguồn ground truth* đo hiệu năng, và phục vụ *truy vấn UTXO* cho client.

Ba nguyên tắc thiết kế xuyên suốt thành phần:

- **Tách trách nhiệm theo gói.** Mỗi mối quan tâm — nhận block, kiểm tra, lưu file, lưu world state, mirror, khám phá orderer, đo lường, mật mã — được đặt trong một gói riêng với giao diện hẹp. Một bộ điều phối trung tâm ghép các gói này lại và định nghĩa luồng dữ liệu giữa chúng.
- **Tách tốc độ bằng kênh.** Các tầng có tốc độ khác nhau (nhận block qua mạng, kiểm tra + ghi đĩa, mirror sang cơ sở dữ liệu) được nối bằng kênh có đệm, để một tầng chậm không ghì lại tầng nhanh.
- **Sổ cái thật không phụ thuộc cơ sở dữ liệu ngoài.** Một block được coi là đã commit ngay khi đã ghi xong file và world state cục bộ; việc ghi PostgreSQL diễn ra bất đồng bộ sau đó và không bao giờ là điều kiện để commit thành công.

3.5.2 Kiến trúc tổ chức theo gói

Toàn bộ thành phần được lắp ráp từ một số gói con, mỗi gói một trách nhiệm rõ ràng. Bảng ?? liệt kê các gói chính và vai trò của chúng, còn Hình ?? minh họa cách chúng ghép nối thành một đường ống.

Table 15. Các gói con của Committing Peer và trách nhiệm

Gói	Trách nhiệm
Bộ điều phối	Lắp ráp mọi gói con, sở hữu kênh nội bộ nối tầng nhận block với vòng commit, giữ số liệu runtime (số block, hash cuối), điều phối vòng commit.
Deliver client	Bọc một host libp2p; đăng ký nhận dòng block từ orderer, đăng ký handler ký giao dịch (endorser) và handler truy vấn UTXO.
Discovery	Cache danh sách thành viên của cluster orderer và chọn orderer còn sống để kết nối lại sau khi Leader đổi.
Validation	Kiểm tra toàn vẹn cấu trúc block (Merkle, hash) và tính hợp lệ endorsement của từng giao dịch hợp đồng.
Block storage	Ghi chuỗi block vào file chỉ-thêm (append-only); là số cái bất biến.
World state	UTXO set trên LevelDB; áp dụng block theo lô nguyên tử.
Postgres mirror	Ghi bản sao block/giao dịch sang PostgreSQL bất đồng bộ, ngoài luồng commit.
Metrics	Ghi thời điểm commit trong RAM và phục vụ API đo throughput/độ trễ.
Crypto	Sinh khóa, ký và xác minh Ed25519; tính lại hash block và gốc Merkle để kiểm tra.

Fig. 6. Kiến trúc đường ống và các gói con của Committing Peer.

Cách ghép nối tuân theo nguyên tắc *tiêm phụ thuộc* (dependency injection): bộ điều phối được khởi tạo với các phụ thuộc đã dựng sẵn (deliver client, engine kiểm tra, block storage, world state, kết nối PostgreSQL). Nhờ vậy mỗi gói con có thể được kiểm thử và suy luận độc lập, còn bộ điều phối chỉ tập trung vào việc định nghĩa *trình tự* và *luồng dữ liệu* giữa chúng, chứ không chứa chi tiết cài đặt của từng tầng.

3.5.3 Đường ống xử lý bằng kênh

Trái tim của thành phần là một đường ống ba chặng, các chặng được tách rời bằng kênh để chạy với tốc độ riêng:

1. **Chặng nhận (producer).** Một goroutine của deliver client đọc liên tục các block từ stream của orderer và đẩy mỗi block vào một *kênh nội bộ* có đệm (sức chứa 64 block). Chặng này chỉ giải mã và chuyển tiếp, không làm việc nặng.

2. **Chặng commit (consumer).** Một vòng lặp commit rút từng block ra khỏi kênh nội bộ và xử lý tuần tự: kiểm tra hợp lệ, ghi file, áp dụng world state, ghi nhận số liệu, rồi đẩy công việc mirror sang chặng ba. Việc xử lý tuần tự ở đây bảo đảm các block được ghi vào sổ cái đúng thứ tự chúng được commit.
3. **Chặng mirror (bất đồng bộ).** Một nhóm worker nền nhận công việc mirror qua một kênh riêng và ghi sang PostgreSQL. Chặng này nằm hoàn toàn ngoài đường commit: dù nó chậm hay lỗi, sổ cái thật vẫn tiến triển bình thường.

Việc dùng kênh có đệm giữa chặng nhận và chặng commit đóng vai một *bộ giảm xóc*: khi orderer phát block dồn dập, kênh hấp thụ các đỉnh tải ngắn để vòng commit xử lý kịp mà không làm nghẽn stream. Nếu tải cao kéo dài và kênh đầy, chặng nhận sẽ tự nhiên bị chặn nhẹ (backpressure) — một cơ chế điều tiết lành mạnh thay vì mất block.

Thứ tự các thao tác trong một lần commit. Với mỗi block, vòng commit thực hiện đúng trình tự sau, và thứ tự này có ý nghĩa về mặt nhất quán:

1. Kiểm tra hợp lệ (Merkle, hash, endorsement). Nếu sai, block bị ghi log và **bỏ qua** — không ghi gì, pipeline tiếp tục với block sau.
2. Ghi block vào file sổ cái (chỉ-thêm).
3. Áp dụng block vào world state (một lô ghi nguyên tử trên LevelDB).
4. Tăng bộ đếm số block và cập nhật số liệu runtime (hash cuối, thời điểm, số giao dịch).
5. Ghi thời điểm commit vào bộ đo trong RAM (phục vụ benchmark).
6. Đẩy một công việc mirror sang chặng ba (không chặn).

Ranh giới “đã commit” được đặt sau bước 3: khi cả file và world state đã ghi xong, block coi như bền vững. Các bước 4–6 chỉ là số liệu và bản sao phụ trợ.

3.5.4 Nhận block và tự phục hồi

Committing Peer nhận block theo mô hình *đăng ký dòng* (streaming subscribe). Khi khởi động, nó mở một stream tới một orderer và gửi kèm một yêu cầu cho biết “*tôi đã có tới block thứ mấy, hãy gửi tiếp từ block kế*”. Chỉ số bắt đầu này được tính bằng **số block đang có trên file cộng một**. Nhờ đó, sau khi khởi động lại, peer không tải lại các block đã có (tránh xử lý trùng) mà cũng không bỏ sót block nào. Orderer đáp lại bằng cách gửi ngay các block còn thiếu rồi tiếp tục đẩy các block tương lai trên cùng stream giữ mở.

Tự kết nối lại sau khi Leader đổi. Vì Leader của cluster orderer có thể thay đổi (do bầu cử lại), stream nhận block có thể đứt bất cứ lúc nào. Một vòng *kết nối lại* thường trực xử lý việc này: khi stream đứt, peer hỏi gói Discovery để tìm một orderer còn sống, rồi mở lại stream với chỉ số bắt đầu được tính lại từ file (luôn là block kế tiếp chưa có) — **không mất block nào**. Để tránh đội kết nối khi cluster đang bất ổn, vòng này dùng *khoảng lùi tăng dần* (exponential backoff): thời gian chờ giữa các lần thử tăng gấp đôi sau mỗi thất bại, từ 1 giây tới tối đa 30 giây, và được đặt lại về 1 giây ngay khi kết nối thành công.

Gói Discovery. Gói khám phá duy trì một bản cache danh sách thành viên của cluster orderer. Nó định kỳ làm mới danh sách này (mỗi vài giây) và cache lại trong một khoảng ngắn (mặc định 8 giây) để không phải truy vấn liên tục. Khi phát hiện một orderer không còn phản hồi, peer *vô hiệu hóa* cache nóng để lần chọn kế tiếp lấy danh sách mới; nếu tạm thời không truy vấn được cluster, gói vẫn dùng bản tốt cuối cùng còn hiệu lực (trong một cửa sổ vài phút) làm phương án dự phòng. Khi cần chọn orderer để kết nối lại, peer duyệt lần lượt các orderer còn sống theo kiểu xoay vòng, nên qua vài lần thử nó sẽ chạm tới Leader mới mà không cần biết trước ai là Leader. Cơ chế cache cộng dự phòng này giúp peer chịu được việc Leader đổi mà không cần cấu hình lại hay khởi động lại.

3.5.5 Vai trò endorser và tính hợp lệ endorsement

Ngoài việc nhận block, Committing Peer còn đăng ký một handler để **ký xác nhận** (endorse) giao dịch cho Core Service. Khi Core gửi một giao dịch vừa “chạy thử” xong, peer thực hiện: xác minh các chữ ký endorsement đã có (nếu có) là hợp lệ; ký thêm chữ ký của mình; tự kiểm tra lại chữ ký vừa tạo; rồi trả giao dịch đã có endorsement về cho Core. Chỉ khi tự kiểm tra thành công peer mới trả kết quả — nó không bao giờ phát ra một chữ ký mà chính nó không xác minh được.

Thông điệp được ký. Điểm thiết kế quan trọng để cả hệ thống nhất quán là *bố cục thông điệp ký* giống hệt nhau ở mọi tầng: chữ ký được đặt trên chuỗi ghép txid + tên hợp đồng + payload. Nhờ layout thống nhất này, chữ ký do Committing Peer tạo lúc endorse có thể được chính nó (hoặc peer khác) xác minh lại y hệt lúc kiểm tra block, không phụ thuộc vào thứ tự trường hay cách tuần tự hóa JSON. Thuật toán ký là **Ed25519**: khóa riêng và khóa công khai

biểu diễn dưới dạng hex, chữ ký 64 byte. Mỗi chữ ký được lưu thành một mục endorsement gồm khóa công khai và chữ ký; nếu peer ký lại một giao dịch mà mình đã ký trước đó, mục cũ được thay thế thay vì thêm trùng.

Tập khóa tin cậy. Peer được cấu hình một *danh sách khóa công khai endorser tin cậy* (các chuỗi hex, phân tách bằng dấu phẩy; nếu không cấu hình thì mặc định tin chính khóa của peer). Danh sách này là gốc tin cậy khi kiểm tra block: một giao dịch hợp đồng chỉ được chấp nhận nếu có ít nhất một endorsement đến từ một khóa nằm trong tập tin cậy. Đây là ranh giới an ninh mấu chốt — Ordering Service không xác minh endorsement, nên toàn bộ trách nhiệm chứng thực rơi vào tầng kiểm tra của Committing Peer.

3.5.6 Kiểm tra hợp lệ block

Trước khi ghi bất kỳ block nào, engine kiểm tra thực hiện hai nhóm kiểm tra.

Toàn vẹn cấu trúc. Engine *tính lại độc lập* hai giá trị rồi so với những gì block mang theo: (i) gốc cây Merkle, tính từ danh sách định danh giao dịch; và (ii) hash của block, tính bằng double-SHA256 trên header (gồm phiên bản, prev hash, gốc Merkle, nhãn thời gian, và nonce). Nếu một trong hai lệch, block bị từ chối. Việc tính lại độc lập bảo đảm block không bị sửa đổi trên đường truyền, và — rất quan trọng — hàm băm cùng cách tính Merkle của Committing Peer phải **trùng khớp tuyệt đối** với cách Ordering Service tạo ra chúng, nếu không một block hợp lệ sẽ bị từ chối oan. Sự trùng khớp này đạt được bằng cách hiện thực cùng một bố cục header kiểu Bitcoin (các trường 32-byte được đệm trái đồng nhất) ở cả hai thành phần.

Kiểm tra endorsement. Nếu có cấu hình tập khóa tin cậy, mỗi giao dịch *hợp đồng* (có payload và tên hợp đồng) trong block phải: có mảng endorsement không rỗng; mỗi endorsement có đủ khóa công khai và chữ ký, và chữ ký xác minh hợp lệ trên thông điệp ký chuẩn; và ít nhất một khóa endorser thuộc tập tin cậy. Giao dịch dạng UTXO thuần (không có payload/hợp đồng) được miễn kiểm tra endorsement. Việc chỉ dựa vào mảng endorsement tường minh (bỏ qua các trường chữ ký kiểu cũ) giữ cho quy tắc kiểm tra rõ ràng và không mơ hồ.

Tính kiên cường. Một block không hợp lệ **không** làm sập pipeline: nó chỉ bị ghi log và bỏ qua, peer tiếp tục xử lý block kế tiếp. Cách xử lý này ưu tiên tính sẵn sàng của dịch vụ ghi sổ trước các block dị thường lẻ tẻ.

Phạm vi kiểm tra hiện tại. Theo mã nguồn, một số kiểm tra ở mức sâu hơn chưa được thực thi trên đường nóng. Việc kiểm tra liên kết chuỗi ở mức peer (prev hash của block khớp hash của block liền trước trên đĩa) hiện được bỏ qua ở engine — tuy prev hash vẫn nằm trong dữ liệu băm để bảo đảm toàn vẹn cấu trúc, và liên kết chuỗi vẫn được xác minh đầy đủ ở luồng đồng bộ của Ordering Service. Kiểm tra hợp lệ ở mức từng giao dịch UTXO (chống tiêu hai lần, xác minh chữ ký đầu vào) hiện để trống. Đây là ranh giới phạm vi của bản hiện thực, cần được nêu rõ để đánh giá đúng mô hình tin cậy: hệ thống dựa vào tính đúng đắn của thứ tự do Ordering Service cung cấp và vào chứng thực endorsement, chứ chưa thực thi đầy đủ ngữ nghĩa UTXO tại tầng commit.

3.5.7 Mô hình lưu trữ ba lớp

Committing Peer tổ chức lưu trữ thành ba lớp tách bạch, mỗi lớp một công nghệ và một mục đích riêng (Bảng ??). Đây là điểm cốt lõi của thiết kế tổ chức thành phần.

Table 16. Ba lớp lưu trữ của Committing Peer

Lớp	Công nghệ	Đặc tính	Mục đích
Block storage	File chỉ-thêm	Bất biến, ghi tuần tự	Sổ cái thật, kiểm toán
World state	LevelDB	Ghi đè, lô nguyên tử	Tra UTXO hiện tại
Mirror	PostgreSQL	Chỉ mục, JSONB, bất đồng bộ	Explorer & báo cáo

Lớp 1 — Block storage (sổ cái bất biến) Chuỗi block được lưu trong một file chỉ-thêm, mỗi block là một dòng JSON. File được mở ở chế độ *chỉ ghi vào cuối* và **không bao giờ** bị cắt ngắn (truncate) — bảo đảm tính “bất biến ở mức file”: một khi đã ghi, block không bị sửa hay xóa. Khi khởi động, peer đọc lại toàn bộ file để khôi phục hai giá trị: *số block đã có* và *hash của block cuối*. Hai giá trị này dùng để tính chỉ số bắt đầu nhận block (Mục ??) và để đồng bộ lại các bộ đếm runtime sau khi khởi động lại, nhờ đó chiều cao block và số liệu tiếp tục đúng. Việc đọc lại dùng bộ đệm lớn (tối 16 MB mỗi dòng) để chứa được các block chứa hàng nghìn giao dịch. Định dạng “một dòng JSON

một block” được chọn vì vừa dễ ghi nối tiếp, vừa đọc được bằng mắt và bằng công cụ, thuận tiện cho kiểm toán.

Lớp 2 — World state (UTXO trên LevelDB) World state là ảnh chụp trạng thái hiện tại: tập các đầu ra chưa tiêu (UTXO). Nó được lưu trên LevelDB theo một lược đồ khóa đơn giản: mỗi UTXO là một cặp khóa `utxo:⟨txid⟩:⟨vout⟩` ánh xạ tới nội dung đầu ra được tuần tự hóa. Khi áp dụng một block, mỗi đầu vào (VIN) *xóa* UTXO mà nó tiêu, mỗi đầu ra (VOUT) *tạo* một UTXO mới.

Điểm thiết kế quan trọng nhất ở lớp này là tính **nguyên tử**: toàn bộ thay đổi của một block — mọi thao tác xóa và thêm — được gom vào *một lô* (batch) và ghi xuống LevelDB trong một thao tác duy nhất kiểu “tất-cả-hoặc-không”. Điều này ngăn tình trạng world state bị sai lệch nếu tiến trình gặp sự cố giữa chừng: sẽ không có chuyện một đầu vào đã bị xóa nhưng đầu ra tương ứng chưa kịp tạo. Nhờ vậy, world state luôn phản ánh đúng kết quả của một tập block trọn vẹn. Lớp này cũng cung cấp các thao tác tra cứu (một UTXO theo cặp txid/chỉ số, liệt kê toàn bộ UTXO, đếm số UTXO) phục vụ truy vấn của client và lệnh quản trị.

Lớp 3 — PostgreSQL mirror (bản sao tra cứu) Lớp thứ ba là một bản sao trên PostgreSQL, phục vụ Explorer và báo cáo. Điểm cốt lõi trong thiết kế: lớp này nằm **hoàn toàn ngoài đường commit**. Sau khi một block đã được ghi file và áp dụng world state (tức đã commit), peer chỉ *đẩy một công việc mirror* vào một kênh rồi tiếp tục ngay với block kế. Một nhóm worker nền (mặc định 2 worker, hàng đợi 512 công việc) nhận và ghi các công việc này sang PostgreSQL. Nếu hàng đợi đầy, peer không chặn đường commit mà sinh một goroutine ghi ngay công việc đó. Mỗi block được ghi trong *một giao dịch cơ sở dữ liệu* gồm một bản ghi block và các bản ghi giao dịch của nó, với ràng buộc chống trùng (nếu block đã tồn tại thì bỏ qua) để việc ghi lại không gây lỗi. Kết nối tới PostgreSQL dùng một pool giới hạn số kết nối để không làm quá tải cơ sở dữ liệu dưới tải cao.

Triết lý. Việc tách mirror ra khỏi đường commit thể hiện rõ nguyên tắc “số cái thật không phụ thuộc cơ sở dữ liệu ngoài”: tốc độ và tính đúng đắn của việc ghi blockchain không bị ràng buộc bởi tốc độ hay tình trạng của PostgreSQL. Nếu PostgreSQL chậm hoặc tạm thời chết, blockchain vẫn tiếp tục commit;

mirror sẽ đuổi kịp sau. Đổi lại, Explorer đọc từ mirror có thể trễ hơn một chút so với thời điểm commit thật — điều chấp nhận được với mục đích khám phá ledger, nhưng vì thế mirror **không** được dùng làm nguồn chuẩn khi đo hiệu năng (xem Mục ??).

3.5.8 Metrics — nguồn ground truth

Vì Committing Peer là nơi giao dịch được commit thật, nó là nguồn “sự thật mặt đất” (ground truth) tốt nhất để đo throughput và độ trễ đầu-cuối. Thiết kế đo lường ở đây có một quyết định quan trọng: thời điểm commit của mỗi giao dịch được ghi **ngay trong bộ nhớ** (một bản đồ định danh giao dịch → thời điểm, kèm danh sách block), *không* đọc lại từ PostgreSQL. Lý do là độ trễ ghi mirror bất đồng bộ sẽ làm sai lệch số đo nếu dùng mốc thời gian từ cơ sở dữ liệu; đo trong RAM cho con số chính xác về đúng thời điểm block được chốt.

Bộ đo tự dọn các bản ghi cũ theo một cửa sổ lưu giữ (mặc định vài giờ) để không phình bộ nhớ trong các phiên benchmark dài. Một máy chủ HTTP nhẹ (mặc định cổng :8081) mở các endpoint đo throughput, tổng hợp benchmark, và tra cứu thời điểm commit theo định danh giao dịch; Core Service gọi các endpoint này để tổng hợp số liệu đầu-cuối. Vì thời điểm commit được ghi độc lập với tầng ký và tầng mirror, số đo giữ nguyên ý nghĩa bất kể cấu hình các tầng đó.

3.5.9 Phục vụ truy vấn UTXO cho client

Committing Peer còn mở một giao thức để client (ví UTXO) truy vấn số dư. Khi nhận một yêu cầu kèm địa chỉ ví, peer duyệt world state và trả về mọi UTXO có địa chỉ người nhận khớp với địa chỉ được hỏi. Đây là cách một ví ngoài đồng bộ tập UTXO khả dụng của mình trực tiếp từ sổ cái đã chốt, thay vì phải tự dựng lại trạng thái từ toàn bộ lịch sử block. Chức năng này tái dùng chính host libp2p của deliver client (một handler giao thức khác trên cùng host), nên không cần thêm cổng mạng riêng.

3.5.10 Khởi động và cấu hình

Trình tự khởi động của peer thể hiện rõ cách các gói được lắp ráp: (1) nạp hoặc sinh khóa endorsement (ưu tiên biến môi trường, rồi tới file, cuối cùng mới

sinh mới và lưu lại); (2) mở block storage và world state; (3) kết nối PostgreSQL cho mirror; (4) dựng deliver client và đăng ký handler ký giao dịch; (5) dựng engine kiểm tra với tập khóa tin cậy; (6) lắp bộ điều phối từ các phụ thuộc trên và đăng ký handler truy vấn UTXO; (7) bật gói Discovery với vòng làm mới định kỳ; (8) tính chỉ số bắt đầu nhận block từ số block đang có trên đĩa rồi khởi động đường ống; và (9) bật máy chủ metrics. Nhiều tham số vận hành được điều khiển qua biến môi trường (Bảng ??), cho phép tinh chỉnh mà không cần biên dịch lại.

Table 17. Một số biến môi trường cấu hình Committing Peer

Biến	Mặc định	Tác dụng
TRUSTED_ENDORSER_PUBLIC_KEYS	khóa của peer	Danh sách khóa endorser tin cậy khi kiểm tra block
POSTGRES_URL	default nội bộ	Chuỗi kết nối PostgreSQL cho mirror
COMMIT_PEER_METRICS_ADDR	:8081	Cổng máy chủ metrics (0 để tắt)
COMMIT_PEER_PG_MIRROR	bật	Bật/tắt mirror sang PostgreSQL
COMMIT_PEER_PG_WORKERS	2	Số worker ghi mirror
COMMIT_PEER_PG_QUEUE	512	Sức chứa hàng đợi mirror
COMMIT_PEER_METRICS_RETENTION	vài giờ	Cửa sổ lưu giữ số liệu commit trong RAM

3.5.11 Mô hình đồng thời và an toàn luồng

Vì Committing Peer xử lý nhiều luồng công việc song song — nhận block qua mạng, commit tuần tự, ghi mirror nền, ký giao dịch cho Core, phục vụ truy vấn UTXO, làm mới danh sách orderer — thiết kế đồng thời cần được tổ chức cẩn thận để vừa an toàn vừa không tạo nút thắt. Cách tổ chức tuân theo hai ý tưởng bổ trợ nhau: *dùng kênh làm ranh giới giữa các luồng* và *dùng khóa cục bộ, phạm vi hẹp cho trạng thái chia sẻ*.

Kênh làm ranh giới. Hai điểm nối quan trọng nhất giữa các luồng là kênh: kênh nội bộ nối chặng nhận block với vòng commit, và kênh công việc nối vòng commit với các worker mirror. Nhờ giao tiếp qua kênh thay vì cùng thao tác trên một vùng nhớ, các chặng gần như không cần khóa chung: mỗi block đi qua đường ống theo kiểu “trao tay” (một producer, một consumer cho mỗi kênh), loại bỏ phần lớn nguy cơ tranh chấp. Vòng commit là điểm tuần tự hóa duy

nhất cho việc ghi sổ, nên thứ tự block và tính nhất quán của world state được bảo đảm một cách tự nhiên mà không cần đồng bộ hóa phức tạp.

Khóa cục bộ, phạm vi hẹp. Với các trạng thái buộc phải chia sẻ giữa nhiều luồng, mỗi trạng thái được bảo vệ bằng một cơ chế riêng, đủ hẹp để không cản trở đường nóng (Bảng ??). Lớp ghi file dùng một khóa riêng bảo vệ thao tác ghi nối và hai giá trị suy ra (số block, hash cuối). World state dựa vào bảo đảm nội tại của LevelDB cho một lô ghi nguyên tử, nên không cần khóa ở tầng ứng dụng. Bộ đếm số block dùng thao tác nguyên tử (atomic) thay vì khóa, vì nó được đọc/ghi rất thường xuyên. Các số liệu runtime hiển thị (hash cuối, thời điểm, số giao dịch của block cuối) được bảo vệ bằng một khóa đọc-ghi, cho phép nhiều luồng đọc trạng thái đồng thời trong khi vòng commit thỉnh thoảng cập nhật. Bộ đo trong RAM có khóa riêng của nó. Việc phân tách khóa theo trách nhiệm như vậy giúp một luồng chậm (ví dụ một truy vấn trạng thái) không chặn đường commit.

Table 18. Trạng thái chia sẻ và cơ chế an toàn luồng tương ứng

Trạng thái	Cơ chế bảo vệ	Lý do
Ghi file + số block/hash cuối trên đĩa	Khóa riêng của lớp ghi file	Ghi nối phải tuần tự
World state (UTXO)	Lô ghi nguyên tử của LevelDB	Tất-cả-hoặc-không cho mỗi block
Bộ đếm số block đã commit	Thao tác nguyên tử (atomic)	Đọc/ghi rất thường xuyên
Số liệu runtime hiển thị	Khóa đọc-ghi	Nhiều đọc, ít ghi
Bộ đo commit trong RAM	Khóa riêng của bộ đo	Tách khỏi đường ghi sổ
Đường ống block & mirror	Kênh có đệm	Trao tay một chiều, không cần khóa chung

Tắt mượt. Toàn bộ các luồng nền được ràng vào một ngữ cảnh hủy chung: khi peer dừng, ngữ cảnh bị hủy làm các vòng lặp nhận block, commit, mirror và làm mới danh sách orderer thoát trật tự, sau đó các tài nguyên (host libp2p, file sổ cái, world state, kết nối cơ sở dữ liệu) được đóng lại. Nhờ đó không có luồng nào bị bỏ rơi và dữ liệu đang ghi được hoàn tất trước khi thoát.

3.5.12 Ranh giới nhất quán và đánh đổi thiết kế

Kết lại, thiết kế của Committing Peer phản ánh một số ranh giới và đánh đổi có chủ đích:

Định nghĩa “đã commit”. Một block bền vững ngay khi đã ghi file và áp dụng world state, độc lập với PostgreSQL. Ranh giới rõ ràng này giúp lập luận về tính nhất quán và tách tốc độ blockchain khỏi tốc độ cơ sở dữ liệu ngoài.

Nguyên tử ở mức world state, tuần tự ở mức block. World state được cập nhật nguyên tử cho từng block, còn các block được commit tuần tự đúng thứ tự nhận. Hai tính chất này cùng nhau giữ cho trạng thái luôn tương ứng với một tiền tổ hợp lệ của chuỗi.

Mô hình tin cậy. Peer tin thứ tự do Ordering Service cung cấp và chỉ thực thi kiểm tra mật mã (toàn vẹn block, chứng thực endorsement từ khóa tin cậy). Các kiểm tra ngữ nghĩa UTXO ở mức từng giao dịch hiện chưa được thực thi đầy đủ trên đường commit — một ranh giới phạm vi cần nêu rõ khi đánh giá đảm bảo an ninh của hệ thống.

Số đo trong RAM. Metrics ground truth nằm trong bộ nhớ nên chính xác về thời điểm nhưng không bền qua khởi động lại; đây là đánh đổi hợp lý vì mục đích của chúng là benchmark, còn dữ liệu bền phục vụ tra cứu đã có ở mirror.

Kiên cường trước lỗi cục bộ. Block dị thường bị bỏ qua thay vì làm sập tiến trình; stream đứt được kết nối lại tự động không mất block; hàng đợi mirror đầy không chặn đường commit. Các lựa chọn này cùng hướng tới việc giữ dịch vụ ghi sổ luôn tiến triển dưới điều kiện vận hành thực tế.

3.6 Blockchain Explorer — Giao diện

3.6.1 Tech stack và kiến trúc

Blockchain Explorer (`BlockchainExplorer-FrontEnd/`) là một ứng dụng web một trang (SPA) viết bằng React, có hai chức năng: khám phá (xem block và giao dịch đã commit theo thời gian thực) và gửi giao dịch mới. Nó nói chuyện với Core Service (`HTTP :8080`) qua proxy của Vite. Bảng ?? liệt kê các công nghệ chính.

Table 19. Công nghệ của Blockchain Explorer

Thư viện	Phiên bản	Vai trò
React	18.3.1	Framework UI (hooks)
React Router DOM	6.26.2	Điều hướng client-side
Vite	5.4.1	Build tool & dev server (proxy /api)
Material-UI (MUI)	6.1.1	Bộ component giao diện
Tailwind CSS	3.4.13	CSS tiện ích
CryptoJS	4.1.1	Băm SHA-256 (dữ liệu mock MockData.js)

Ứng dụng dùng React Router với ba tuyến đường (`/transactions`, `/transfer`, `/blocks`), tất cả render qua Dashboard với prop `section` khác nhau. Lớp gọi API (`src/api/client.js`) dùng URL tương đối được Vite proxy chuyển tới Core Service, tránh vướng CORS khi dev.

Explorer **không** đọc trực tiếp `chain.block` hay LevelDB. Danh sách block và giao dịch trên UI đều qua Core API; Core đọc bản mirror trong schema `commit_peer` của PostgreSQL. Mirror ghi bắt đồng bộ sau khi commit peer ghi file, nên UI có thể chậm hơn một chút so với thời điểm commit thật — đủ cho khám phá ledger, không dùng làm ground truth khi benchmark.

3.6.2 Mã hóa payload nhị phân

Payload của giao dịch cần nhỏ gọn và đúng kiểu để hợp đồng đọc. Frontend có hai cơ chế mã hóa, đều xuất ra chuỗi hex: (1) **hệ nhị phân tùy biến** (`transactionTypes.js`) đóng gói từng trường thành byte (big-endian, với các kiểu `STRING`, `UINT8/16/32/64`, `FLOAT/DOUBLE`, `BYTES`, `BOOLEAN`, `ADDRESS`), tiết kiệm $\sim 70\%$ so với JSON; và (2) **hệ payload contract** (`transactionUtils.js`) `JSON` $\rightarrow$ `UTF-8` $\rightarrow$ `hex`, để contract Go dùng `json.Unmarshal` đọc.

Vì contract WASM hiện đọc payload bằng `json.Unmarshal`, form Transfer dùng cơ chế JSON-hex khi gửi tới contract. Codec nhị phân chỉ còn trong decoder legacy khi đọc payload cũ. Hàm `coercePayloadFieldsBySchema` ép kiểu input HTML (luôn là chuỗi) về int/float đúng theo schema trước khi mã hóa. `txid` sinh bằng `crypto.randomUUID()` phía client (không qua CryptoJS). Sau submit thành công, UI không cập nhật danh sách ngay mà gọi lại API — chỉ hiện giao dịch đã xuất hiện trên mirror Postgres.

3.6.3 Cập nhật realtime qua SSE

Explorer dùng SSE để hiện block/giao dịch mới ngay khi commit (qua mirror). Khi Dashboard mount, nó mở `EventSource('/api/explorer/stream')`; nhận sự kiện `ledger_update` (chứa `latest_block` và `latest_tx`) thì cập nhật (upsert) danh sách. Phía Core, `HandleExplorerStream` poll PostgreSQL mỗi 2 giây rồi emit khi phát hiện block hoặc tx mới — kéo có nhịp qua mirror DB, không phải push trực tiếp từ commit peer.

Nếu SSE đứt, Dashboard chuyển sang **polling dự phòng** (gọi `/api/transactions` và `/api/blocks` mỗi 3000 ms), tự reconnect SSE sau 3 giây, và dừng polling khi stream kết nối lại — mẫu “ưu tiên đẩy, có lưới đỡ kéo”. Trạng thái kết nối được hiển thị bằng chữ và màu (`connected/connecting/disconnected`).

3.7 Cơ sở dữ liệu PostgreSQL

Cần nhấn mạnh lại: PostgreSQL **không** nằm trong luồng đồng thuận; số cái thật là `chain.block` + LevelDB ở Committing Peer. PostgreSQL là bản sao phục vụ hiển thị Explorer, lưu danh mục contract và đo benchmark. Dữ liệu được tách thành **ba schema** theo đúng ranh giới trách nhiệm của ba dịch vụ, như Bảng ??.

Table 20. Ba schema của cơ sở dữ liệu

Schema	Ai ghi	Nội dung
<code>core_service</code>	Core Service	Contract, thời điểm submit
<code>order_service</code>	Orderer (tùy chọn)	Giao dịch & block đã sắp xếp
<code>commit_peer</code>	Committing Peer	Số cái đã commit (mirror)

3.7.1 Các bảng chính

`core_service.smart_contracts`: lưu bytecode WASM (`contract_code`), tên contract (UNIQUE), `payload_schema` JSONB (tùy chọn) và timestamp. Core ghi khi deploy qua `POST /api/tx/deploy`.

`core_service.tx_submit_times`: `txid` PRIMARY KEY, `submitted_at` TIMESTAMPTZ, index theo `submitted_at` DESC. Core ghi khi accept HTTP submit (`CORE_RECORD_SUBMIT=1`). Dùng đo submit và join E2E — không đồng nghĩa tx đã commit.

`commit_peer.ledger`: `block_hash` UNIQUE, `block_number`, `block_data` JSONB, `committed_at`. Commit peer mirror sau khi ghi file và world state.

`commit_peer.ledger_transactions`: liên kết `block_id` → `ledger.id`, `txid`, `tx_index`, `tx_data` JSONB (payload hex, endorsements, `algorithm`), khóa ngoại ON DELETE CASCADE, UNIQUE (`block_id`, `txid`).

`order_service`: bảng `transactions`, `blocks`, `block_transactions` — mirror tùy chọn từ orderer; Explorer hiện không đọc schema này.

3.7.2 Ghi mirror và đo benchmark

UTXO world state **không** ở PostgreSQL — nó nằm trên đĩa (LevelDB) tại Committing Peer. Các thao tác chèn mirror dùng ON CONFLICT DO NOTHING để idempotent. Hàng đợi mirror mặc định 512 slot, 2 worker; queue đầy thì spawn goroutine ghi ngay — commit blockchain không chờ Postgres.

Khi đo hiệu năng chính xác, benchmark API ưu tiên thời điểm commit từ commit peer (:8081) thay vì chỉ dựa vào `committed_at` trên mirror, vì mirror có độ trễ so với sổ cái thật.

3.8 Phương pháp đo lường hiệu năng

3.8.1 Ba lớp đo lường

Một giao dịch đi qua nhiều chặng; đo ở chặng khác nhau cho con số khác nhau. Khóa luận phân biệt rõ ba lớp đo (Bảng ??).

Table 21. Ba lớp đo lường hiệu năng

Lớp	Nguồn	Đo cái gì
k6 (client)	Script <code>k6/submit-tx.js</code>	Độ trễ HTTP, Core có trả 200 không
Submit (Core)	<code>core_service.tx_submit_time</code>	Thời gian Core <i>chấp nhận</i> giao dịch
Commit (ledger)	<code>commit_peer.ledger*</code>	Lúc giao dịch <i>đã ghi sổ</i>

E2E latency = `committed_at` − `submitted_at` (ghép theo `txid`). Cần lưu ý `submit_latency_ms` của k6 (vài chục ms) **khác** `latency_ms_p95` E2E (có thể vài giây khi có backlog): một cái đo “Core nhận”, một cái đo “cả pipeline”.

3.8.2 Hai chỉ số cốt lõi

Throughput (TPS): số giao dịch/giây, phân biệt *sustained* (bền vững — trung bình cả cửa sổ) và *peak* (đỉnh — nhiều nhất trong một giây). **Latency (độ trễ):** đo bằng phân vị p50/p95/p99 (p95 nghĩa là 95% giao dịch nhanh hơn giá trị này). Hai chỉ số này thường đánh đổi nhau.

3.8.3 Hiện tượng backlog và định luật Little

Khi **tốc độ gửi** > **tốc độ commit bền vững**, giao dịch dồn vào TxPool và backlog tích lũy, làm E2E latency tăng dù Core chấp nhận nhanh — biểu hiện kinh điển của hệ thống đạt **điểm bão hòa** (saturation). Hiện tượng này được lý giải bằng **định luật Little** [?]: số việc tồn đọng trung bình $L = \lambda W$, với λ là tốc độ đến và W là thời gian lưu trú — khi λ vượt năng lực xử lý, W (và do đó độ trễ) tăng không giới hạn.

3.8.4 Kết quả thực nghiệm tham khảo

Trong phép đo tham khảo ($5000 \text{ req/s} \times 60\text{s}$), hệ thống đạt các con số trong Bảng ??.

Table 22. Kết quả benchmark tham khảo (5000 req/s, 60s)

Chỉ số	Giá trị
Submit sustained	$\sim 4.691/\text{s}$ (đỉnh $\sim 5.017/\text{s}$)
Commit sustained	$\sim 3.921/\text{s}$ (đỉnh $\sim 6.000/\text{s}$)
Trung bình tx/block	~ 989 (batch ~ 1.000)
E2E p95	$\sim 7,4\text{s}$ (khi gửi > commit $\rightarrow$ backlog)

Diễn giải: Core nhận gần đủ 5000/s, nhưng commit “trần” $\sim 3.900/\text{s}$ bền vững, nên chênh lệch $\sim 800/\text{s}$ tạo backlog và đẩy p95 lên mức giây. Nguyên nhân gốc là giới hạn “1 block in-flight” (Mục ??): chu trình propose $\rightarrow$ ACK $\rightarrow$ commit còn tuần tự, nên throughput bị chặn ở mức $1/RTT\text{-commit}$. Đây chính là động lực cho hướng cải thiện quan trọng nhất: **pipeline nhiều block in-flight** — sẽ được phân tích ở chương đề xuất cải thiện.

3.9 Tổng kết công nghệ

Bảng ?? tổng kết ánh xạ giữa các vấn đề kỹ thuật và công nghệ được chọn để giải quyết, khép lại bức tranh thiết kế tổng thể.

Table 23. Ánh xạ “vấn đề ↔ công nghệ”

Vấn đề	Công nghệ
Các node nói chuyện an toàn	libp2p + Ed25519
Đồng ý cùng một thứ tự giao dịch	Raft (tự cài)
Chạy logic nghiệp vụ an toàn, đa ngôn ngữ	WASM (wazero + TinyGo)
Lưu trạng thái nhanh, nhúng sẵn	LevelDB
Dữ liệu không thể sửa	File append-only + hash chain
Tra cứu & báo cáo linh hoạt	PostgreSQL
Hiển thị realtime cho người dùng	React + SSE
Đo hiệu năng dưới tải	k6 + loadgen

Toàn bộ hệ thống được hiện thực bằng Go (ba dịch vụ backend) và React (giao diện), giao tiếp qua libp2p, với Raft tự cài đặt làm trái tim đồng thuận. Thiết kế tách tầng Execute–Order–Validate, kết hợp các tối ưu hot-path và cơ chế mirror bất đồng bộ, cho phép hệ thống tiệm cận mục tiêu 5.000 TPS trong khi vẫn giữ được tính module hóa và khả năng chịu lỗi — đồng thời bộc lộ rõ các điểm đánh đổi và hạn chế làm nền cho các đề xuất cải thiện ở những chương tiếp theo.

Chapter 4

Thực nghiệm và Đánh giá hiệu năng

Chương này trình bày quá trình thực nghiệm đo hiệu năng của Ordering Service — thành phần trọng tâm của hệ thống và cũng là nơi quyết định trần thông lượng đóng block. Mục tiêu là (i) xác định *điểm bão hòa* của dịch vụ, (ii) đo ảnh hưởng của từng **tham số cấu hình** (kích thước block, chu kỳ đề xuất block) và của từng **tham số tải** (tốc độ gửi, số luồng gửi), rồi (iii) từ đó rút ra bộ tham số cho hiệu năng tốt nhất. Toàn bộ số liệu trong chương được sinh ra bằng công cụ `loadgen` đã mô tả ở Chương 3 (Mục ??), chạy trên cùng một máy để bảo đảm tính so sánh được.

4.1 Mục tiêu và câu hỏi thực nghiệm

Ở Mục ??, khóa luận đã phân biệt ba lớp đo: lớp client (k6), lớp Core (submit), và lớp ledger (commit). Chương này tập trung hoàn toàn vào **lớp đóng block của Ordering Service** — tức tốc độ mà leader lấy giao dịch khỏi TxPool, tạo block, đạt đồng thuận và phát block đã commit qua deliver stream. Việc cô lập đúng lớp này cho phép trả lời các câu hỏi sau mà không bị nhiễu bởi WASM, ký endorsement hay độ trễ mirror PostgreSQL:

- **Q1 — Điểm bão hòa.** Khi tăng dần tốc độ gửi, tại đâu tốc độ commit ngừng bám theo tốc độ gửi? Trần thông lượng đóng block là bao nhiêu?
- **Q2 — Kích thước block.** Tham số `auto_propose_block_size` ảnh hưởng thế nào tới thông lượng (tx/s) và tần suất đóng block (blocks/s)?
- **Q3 — Chu kỳ đề xuất block.** Tham số `auto_propose_interval` (block poll) có vai trò gì, và trong điều kiện tải nào nó thực sự chi phối?
- **Q4 — Số luồng gửi.** Tham số `-workers` của `loadgen` ảnh hưởng ra sao tới thông lượng đo được?
- **Q5 — Cấu hình tốt nhất.** Bộ tham số nào cho thông lượng cao và ổn định, và nó đánh đổi gì với độ trễ đóng block?

4.2 Thiết lập thực nghiệm

4.2.1 Môi trường và công cụ

Toàn bộ phép đo chạy trên một máy đơn (single-host), loadgen và orderer giao tiếp qua libp2p trên giao diện loopback. Cấu hình môi trường được liệt kê ở Bảng ??.

Table 24. Môi trường thực nghiệm

Thành phần	Chi tiết
CPU	Intel Core Ultra 7 255H (16 nhân)
Bộ nhớ	16 GB RAM
Hệ điều hành	Windows 11 (64-bit)
Runtime	Go 1.25
Mạng	libp2p trên loopback (không độ trễ mạng)
Công cụ đo	loadgen (giao thức endorsement + deliver stream)

loadgen gửi các giao dịch hợp đồng giả lập (**bench_ping**) trực tiếp tới orderer qua đúng giao thức endorsement mà Core Service dùng, đồng thời subscribe deliver stream để đếm số block và số giao dịch được commit. Chỉ số báo cáo chính là **During load** — số giao dịch/block được commit *trong đúng của số gửi tải*, phản ánh thông lượng bền vững của orderer; các block commit muộn trong pha drain không được tính vào chỉ số này để tránh làm loãng con số.

4.2.2 Mô hình một node và ý nghĩa của phép đo

Các phép đo dùng một cluster **một node** (leader tự đạt đa số — self majority), nên bước đồng thuận commit diễn ra tức thời. Lựa chọn này là có chủ đích: nó *cô lập trần của đường ống đóng block* (TxPool → đề xuất → tính Merkle/hash → commit → deliver) khỏi độ trễ vòng ACK giữa nhiều node. Vì hai tham số trọng tâm cần tinh chỉnh — kích thước block và chu kỳ đề xuất — chi phối cơ chế gom lô (batching) bất kể số node, đo trên một node cho tín hiệu sạch nhất về tác động của chúng. Trong cụm nhiều node, thông lượng sẽ thấp hơn do cộng thêm một số hạng RTT đồng thuận gần như cố định; con số một node vì vậy đóng vai **cận trên** của năng lực lớp ordering (xem bàn luận ở Mục ??).

Do các phép đo throughput nhạy với nhiễu (GC, lập lịch của hệ điều hành, chia sẻ CPU), mỗi cấu hình được chạy lặp lại **ba lần** và báo cáo theo giá trị

trung vị (median) để giảm ảnh hưởng của các lần chạy dị thường. Mỗi lần chạy gồm 15 s gửi tải và 8 s drain.

4.2.3 Ma trận tham số

Bốn nhóm thí nghiệm (sweep) được thiết kế để tách biệt tác động của từng tham số (Bảng ??). Trong mỗi nhóm, chỉ một tham số thay đổi, các tham số còn lại được giữ cố định.

Table 25. Ma trận tham số thực nghiệm (mỗi cấu hình chạy 3 lần)

Nhóm	Tham số quét	Giữ cố định
Tải gửi (TPS)	$\text{tps} \in \{2, 5, 10, 15, 20\}k, \infty$	$\text{block}=1000, \text{iv}=100\text{ms}, \text{w}=32$
Kích thước block	$\text{block} \in \{100, 250, 500, 1000, 2000, 4000\}$	$\text{tps}=\infty, \text{iv}=100\text{ms}, \text{w}=32$
Chu kỳ đề xuất	$\text{iv} \in \{20, 50, 100, 200, 500\}\text{ms}$	$\text{block}=1000, \text{tps}=5000, \text{w}=32$
Số luồng gửi	$\text{w} \in \{8, 16, 32, 48, 64\}$	$\text{block}=500, \text{tps}=\infty, \text{iv}=100\text{ms}$

Lưu ý: nhóm “chu kỳ đề xuất” được đo ở *tải dưới bão hòa* ($\text{tps}=5000$) — đây là điều kiện duy nhất mà chu kỳ đề xuất thực sự chi phối nhịp đóng block (giải thích ở Mục ??); và nhóm “số luồng gửi” được đo tại $\text{block}=500$ — một điểm bão hòa *ổn định* thay vì $\text{block}=1000$ (lý do ở Mục ??).

4.3 Kết quả

4.3.1 Q1 — Điểm bão hòa theo tốc độ gửi

Bảng ?? và Hình ?? trình bày kết quả khi tăng dần tốc độ gửi ở cấu hình mặc định ($\text{block}=1000, \text{iv}=100\text{ms}, 32$ luồng). Trong mọi lần chạy, số giao dịch gửi lỗi (**failed**) đều bằng 0.

Table 26. Ảnh hưởng của tốc độ gửi (block=1000, iv=100ms, 32 luồng)

TPS đặt Gửi (tx/s)	Commit (tx/s)	Blocks/s	Tx/block
2.000	1.987	1.986	9,87
5.000	4.984	4.965	9,86
10.000	9.933	9.864	9,87
15.000	14.915	14.855	14,86
20.000	19.866	19.857	19,87
∞	73.775	73.509	73,51

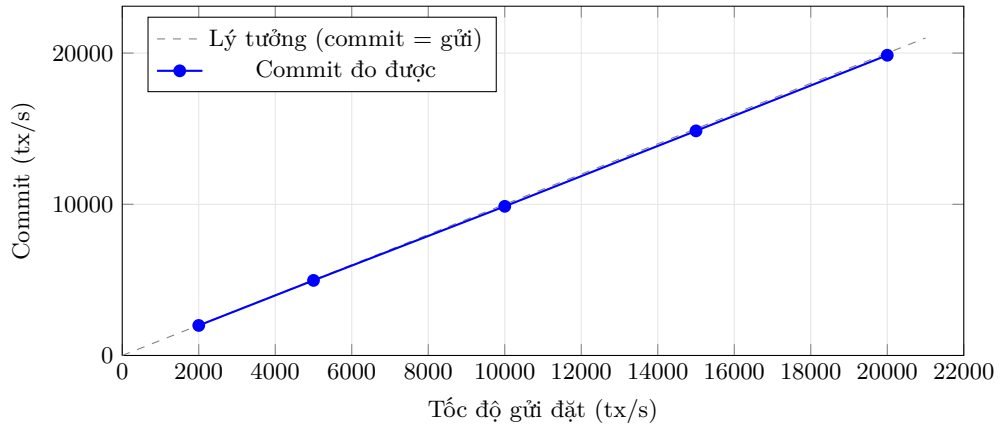


Fig. 7. Tốc độ commit bám sát tốc độ gửi tới ít nhất 20.000 tx/s; trần thực đo ở tải không giới hạn là ~ 73.500 tx/s.

Nhận xét. Trong khoảng 2.000–20.000 tx/s, tốc độ commit gần như trùng tốc độ gửi (sai lệch $< 1\%$): orderer xử lý kịp, chưa bão hòa. Điểm đáng chú ý là **tần suất đóng block** bị ghim ở $\sim 9,87$ blocks/s khi $\text{tps} \leq 10\text{k}$. Nguyên nhân là cơ chế đề xuất theo sự kiện (event-driven) mô tả ở Chương 3: khi TxPool chưa gom đủ một lô đầy (1.000 tx) trong một chu kỳ, leader chờ hết chu kỳ 100ms rồi mới flush — cho đúng $1/0,1 = 10$ block mỗi giây. Vì thế số tx mỗi block tăng tuyến tính theo tải ($201 \rightarrow 503 \rightarrow 997$) trong khi blocks/s giữ nguyên. Khi tải vượt ngưỡng lấp đầy lô ($\geq 15\text{k}$), pool đầy trước 100ms nên leader đề xuất ngay, blocks/s bắt đầu tăng (14,86 rồi 19,87). Ở tải không giới hạn, hệ thống đạt trần ~ 73.500 tx/s tương ứng $\sim 73,5$ block/s — đây là năng lực đóng block thuần của một leader trên môi trường thử nghiệm.

4.3.2 Q2 — Ảnh hưởng của kích thước block

Bảng ?? và Hình ?? đo thông lượng ở tải bão hòa (tps không giới hạn) khi thay đổi kích thước block.

Table 27. Ảnh hưởng của kích thước block (tps= ∞ , iv=100ms, 32 luồng)

Block size	Commit (tx/s)	Blocks/s	Tx/block
100	74.546	745,5	100
250	82.497	330,0	250
500	77.774	155,6	500
1.000	57.439	57,4	1.000
2.000	67.307	33,7	2.000
4.000	73.972	18,5	3.999

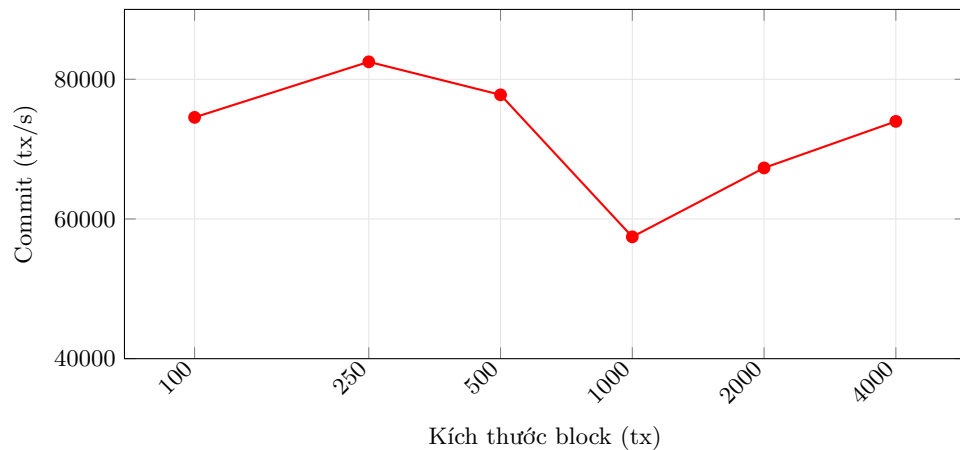


Fig. 8. Thông lượng commit theo kích thước block: đạt đỉnh ~ 82.500 tx/s ở block=250, với một vùng trùng lặp lại được ở block=1.000.

Nhận xét. Tần suất đóng block tỉ lệ nghịch với kích thước block (blocks/s $\times$ block size $\approx$ tx/s), đúng như kỳ vọng. Về thông lượng, tx/s giữ ở mức cao ($\sim 67\text{--}82\text{k}$) trên phần lớn dải, **đạt đỉnh ~ 82.500 tx/s tại block=250**. Đáng chú ý, block=1.000 — giá trị mặc định — lại là một **vùng trùng lặp lại được** (~ 57.400 tx/s, thấp hơn hai lân cận), và các lần chạy tại điểm này cũng *dao động mạnh nhất* (36k–83k giữa các lần đo). Điều này gợi ý block=1.000 nằm gần một *ranh giới bất ổn*: tốc độ lấp đầy lô dao động quanh đúng ngưỡng batch nên vòng đề xuất lúc thì chờ hết chu kỳ, lúc thì flush ngay — tạo hành vi hai chế độ (bimodal). Các kích thước nhỏ hơn (250–500) hoặc lớn hơn (4.000) đều ổn

định và cho thông lượng cao hơn. Đây là một kết quả thực nghiệm quan trọng: **giá trị mặc định 1.000 vừa không tối ưu về thông lượng vừa kém ổn định.**

4.3.3 Q3 — Vai trò của chu kỳ đề xuất block

Cần phân biệt hai chế độ. Ở *tải bão hòa*, TxPool luôn có sẵn hơn một lô đầy, nên vòng đề xuất theo sự kiện bỏ qua thời gian chờ chu kỳ — do đó chu kỳ đề xuất **gần như không tác động** tới thông lượng (thí nghiệm đo tại block=1.000, tps= ∞ cho các giá trị dao động 56k–76k không theo xu hướng rõ ràng, đúng với dự đoán rằng chu kỳ bị bỏ qua). Vai trò thực của chu kỳ chỉ lộ ra ở *tải dưới bão hòa*, khi pool chưa gom đủ lô: khi đó chu kỳ quyết định nhịp đóng block và độ mịn gom lô. Bảng ?? đo ở tps=5.000 (dưới bão hòa).

Table 28. Ảnh hưởng của chu kỳ đề xuất ở tải dưới bão hòa (tps=5.000, block=1.000, 32 luồng)

Chu kỳ (ms)	Commit (tx/s)	Blocks/s	Tx/block
20	4.966	15,19	327
50	4.966	11,87	418
100	4.964	9,80	507
200	4.932	4,93	1.000
500	4.800	4,80	1.000

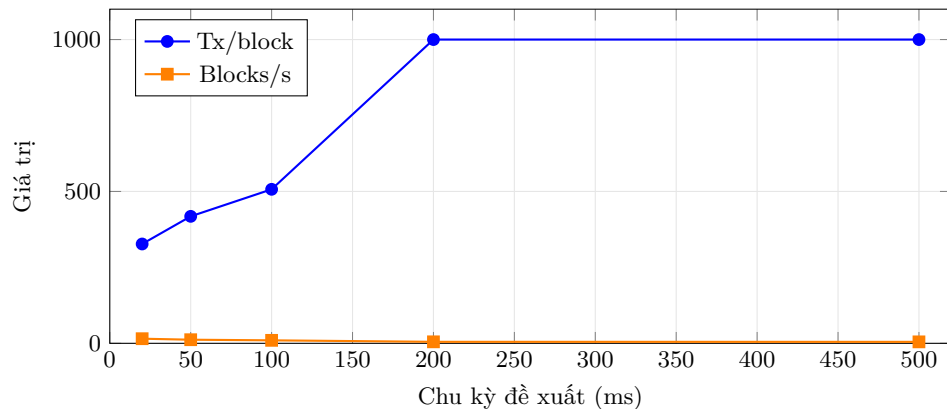


Fig. 9. Ở tải 5.000 tx/s: chu kỳ ngắn cho nhiều block nhỏ, chu kỳ dài cho ít block lớn (bão hòa ở kích thước lô 1.000); thông lượng commit không đổi ~5.000 tx/s.

Nhận xét. Ở tải 5.000 tx/s, tốc độ commit giữ nguyên ~ 5.000 tx/s với mọi chu kỳ (orderer luôn theo kịp), nhưng **tần suất đóng block và số tx/block thay đổi rõ rệt**: chu kỳ ngắn cho nhiều block nhỏ (độ trễ đóng block thấp), chu kỳ dài cho ít block lớn (độ trễ cao hơn nhưng ít overhead trên mỗi giao dịch). Nói cách khác, chu kỳ đề xuất là **núm điều chỉnh đánh đổi độ trễ $\leftrightarrow$ độ mịn gom lô**, không phải núm điều chỉnh thông lượng.

4.3.4 Q4 — Ảnh hưởng của số luồng gửi

Để tách tác động của số luồng gửi khỏi sự bất ổn tại block=1.000, thí nghiệm này đo tại điểm bão hòa ổn định block=500. Kết quả ở Bảng ?? và Hình ??.

Table 29. Ảnh hưởng của số luồng gửi (block=500, tps= ∞ , iv=100ms)

Số luồng Commit (tx/s) Blocks/s		
8	80.196	160,4
16	81.590	163,2
32	82.780	165,6
48	63.963	127,9
64	65.984	132,0

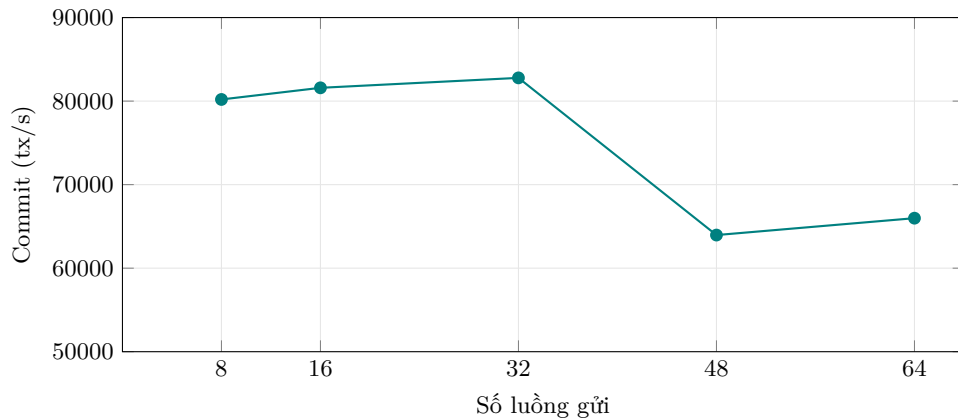


Fig. 10. Thông lượng theo số luồng gửi: vùng phẳng tối ưu ở 8–32 luồng (đỉnh tại 32, ~ 82.800 tx/s), suy giảm khi vượt 32 do tranh chấp stream.

Nhận xét. Số luồng gửi ảnh hưởng tới *tốc độ nạp* (ingest) chứ không phải năng lực đóng block của orderer; tuy nhiên nếu quá ít luồng thì không bơm đủ tải để orderer bão hòa, còn quá nhiều luồng lại gây tranh chấp (mỗi luồng gửi

một stream libp2p) và làm giảm thông lượng đo được. Kết quả cho thấy một vùng tối ưu ở giữa; ngoài vùng này thông lượng suy giảm.

4.4 Cấu hình khuyến nghị

Tổng hợp bốn nhóm thí nghiệm, bộ tham số cho hiệu năng cao và ổn định được đề xuất ở Bảng ??.

Table 30. Bộ tham số khuyến nghị cho Ordering Service

Tham số	Khuyến nghị	Lý do
<code>auto_propose_block_size</code>	250–500	Thông lượng đỉnh (~ 82.500 tx/s ở 250), ổn định; tránh vùng trùng bắt ổn ở 1.000
<code>auto_propose_interval</code>	100ms	Cân bằng độ trễ/gom lô ở tải thấp; không ảnh hưởng khi bão hòa
Số luồng gửi (loadgen)	32	Đủ bơm bão hòa mà chưa gây tranh chấp stream

So với mặc định (block=1.000), chuyển sang block=250–500 vừa **tăng thông lượng** ($\sim 57k \rightarrow \sim 78\text{--}82k$ tx/s, cải thiện $\sim 35\text{--}44\%$) vừa **loại bỏ dao động** của điểm vận hành. Đánh đổi duy nhất là số block nhiều hơn (blocks/s cao hơn), làm tăng chút overhead ở các thành phần hạ nguồn (Committing Peer, mirror PostgreSQL); tuy nhiên các thành phần này được thiết kế bất đồng bộ nên không nằm trên đường tới hạn của việc đóng block.

4.5 Bàn luận và giới hạn

Ordering không phải là nút thắt của hệ thống. Trần đóng block thuần ($\sim 73\text{--}82k$ tx/s) cao hơn nhiều lần so với thông lượng end-to-end tham khảo qua toàn pipeline (~ 3.900 commit/s, Mục ??). Điều này khẳng định lớp ordering *không* phải cổ chai: giới hạn thực tế nằm ở đường Core (WASM + ký endorsement) và ở ràng buộc “một block in-flight” của vòng đồng thuận nhiều node (Mục ??), chứ không ở năng lực gom lô/đóng block của leader.

Giới hạn của phép đo. Các con số trên là **cận trên** của lớp ordering, cần diễn giải trong phạm vi sau: (i) cluster một node nên không có độ trễ ACK đồng

thuận giữa nhiều node; (ii) loopback nên không có độ trễ mạng thực; (iii) trạng thái in-memory nên không có chi phí fsync xuống đĩa; (iv) giao dịch giả lập không thực thi WASM. Trong triển khai nhiều node trên mạng thật, thông lượng sẽ thấp hơn do cộng thêm RTT đồng thuận và chi phí I/O; song thứ hạng tương đối giữa các cấu hình (đặc biệt vùng trùng tại block=1.000 và đỉnh tại block=250–500) được kỳ vọng giữ nguyên vì chúng bắt nguồn từ cơ chế gom lô độc lập với số node. (v) Phép đo thực hiện trên một máy chia sẻ tài nguyên nên còn nhiễu; việc lặp ba lần và lấy trung vị nhằm giảm thiểu ảnh hưởng này.

Chapter 5

Kết luận và Hướng phát triển

Chương này tổng kết những gì khóa luận đã đạt được, thẳng thắn chỉ ra các giới hạn của hệ thống ở trạng thái hiện tại, rồi từ đó đề xuất các hướng phát triển tiếp theo. Các hướng đề xuất không phải là danh sách tính năng ngẫu nhiên, mà bám sát những *ranh giới thiết kế* và *giới hạn của phép đo* đã được nêu ở Chương 3 và Chương ???: mỗi hướng nhằm gỡ bỏ một giả định đã được cố ý đặt ra để giữ phạm vi khóa luận tập trung vào luồng xử lý cốt lõi.

5.1 Kết luận

Khóa luận đã **tìm hiểu, thiết kế và hiện thực từ đầu** một nền tảng blockchain có cấp phép định hướng doanh nghiệp, lấy cảm hứng từ kiến trúc Hyperledger Fabric. Thay vì tái sử dụng một nền tảng có sẵn, chúng tôi tự cài đặt từng thành phần lõi để làm chủ công nghệ: từ tầng mạng ngang hàng `libp2p`, thuật toán đồng thuận Raft, máy ảo hợp đồng thông minh WebAssembly, cho tới tầng sổ cái bất biến và world state UTXO.

Cụ thể, hệ thống hiện thực trọn vẹn triết lý tách ba trách nhiệm **Execute—Order—Validate** qua bốn tiến trình độc lập, với các đóng góp nổi bật: (i) một hiện thực Raft hoàn chỉnh kèm biến thể bầu lãnh đạo theo độ ưu tiên loại bỏ chia phiếu và hội tụ xác định; (ii) một máy ảo WASM với cache module và pool sandbox cho thực thi song song; (iii) một cơ chế cắt block lai giữa kích thước lô và thời gian, kèm giao block theo mô hình đăng-ký-phát-tỏa và đồng bộ bốn pha dựa trên (`commitIndex`, `commitHash`); và (iv) một phương pháp đo lường ba lớp (client, submit, commit) cho phép cô lập và phân tích trần thông lượng của từng tầng.

Về mặt hiệu năng, thực nghiệm ở Chương ?? cho thấy tầng ordering *không* phải là nút thắt: trần đóng block thuần đạt $\sim 73\text{--}82\text{k tx/s}$, cao hơn nhiều lần thông lượng end-to-end tham khảo qua toàn pipeline. Kết quả này vừa xác nhận tính đúng đắn của thiết kế đồng thuận, vừa chỉ rõ cổ chai thực tế nằm ở đường

Core (WASM và ký endorsement) — một tri thức định hướng trực tiếp cho các hướng phát triển bên dưới.

5.2 Giới hạn hiện tại

Ở trạng thái hiện tại, hệ thống còn các giới hạn sau, phần lớn là hệ quả của những giả định được cố ý đặt ra để giữ phạm vi:

- **Mô hình tin cậy dừng lỗi (crash fault tolerance).** Raft chỉ chịu được lỗi *dừng* (crash/mạng phân mảnh), không chịu được node hành xử ác ý (Byzantine). Khóa luận mới chỉ *thảo luận* BFT (Mục 1.5, Chương 2) chứ chưa hiện thực.
- **Ràng buộc “một block in-flight”.** Vòng đồng thuận hiện đề xuất và commit tuần tự từng block một; leader phải chờ block trước được commit mới cắt block kế. Đây là một cận trên tiềm ẩn của thông lượng khi chạy nhiều node trên mạng thật (Mục ??).
- **Trạng thái đồng thuận in-memory, chưa bền vững.** Log Raft và trạng thái bầu cử hiện giữ trong bộ nhớ; chưa có ghi trước (write-ahead log), fsync, snapshot hay nén log. Một node khởi động lại phải dựa vào cơ chế sync để bắt kịp, và cả cluster không sống sót qua sự cố mất điện đồng thời.
- **Thành viên tĩnh.** Danh sách thành viên và độ ưu tiên được cấu hình tĩnh khi khởi động; chưa hỗ trợ thêm/bớt node lúc chạy (dynamic membership reconfiguration).
- **Phép đo là cận trên của một máy đơn.** Số liệu ở Chương ?? đo trên cluster một node, loopback, in-memory, và giao dịch giả lập không thực thi WASM. Chúng chưa phản ánh RTT đồng thuận nhiều node, độ trễ mạng thật và chi phí I/O đĩa.
- **Bảo mật danh tính và kênh truyền còn sơ khai.** Hệ thống chưa có hạ tầng chứng chỉ (MSP/CA) đầy đủ, kênh libp2p chưa siết TLS/xác thực hai chiều bắt buộc, và chưa có phân quyền truy cập ở mức tổ chức.

5.3 Hướng phát triển

5.3.1 Nâng cấp mô hình tin cậy lên chịu lỗi Byzantine

Hướng có giá trị học thuật cao nhất là thay/bổ sung tầng đồng thuận CFT hiện tại bằng một thuật toán **chịu lỗi Byzantine** (BFT). Do kiến trúc đã tách bạch tầng ordering sau một giao diện thông điệp rõ ràng (Mục ?? về mô hình thông điệp và định tuyến), việc thay lỗi đồng thuận không buộc phải viết lại Core hay Committing Peer. Các ứng viên: PBFT [?] cổ điển ($O(N^2)$ thông điệp mỗi vòng), hoặc các biến thể hiện đại theo hướng leader-based tuyến tính hoá chi phí. Việc này nâng mô hình an toàn từ quorum đa số ($\lfloor N/2 \rfloor + 1$) lên ngưỡng $\lfloor (N - 1)/3 \rfloor$ node lỗi, phù hợp với các liên minh doanh nghiệp mà các tổ chức thành viên không hoàn toàn tin nhau.

5.3.2 Bền vững hoá và phục hồi tầng đồng thuận

Bổ sung **write-ahead log** có fsync cho log Raft, kèm cơ chế **snapshot và nén log** để tránh log tăng vô hạn và rút ngắn thời gian phục hồi. Kết hợp với cơ chế sync bốn pha hiện có (Mục ??), một node có thể khôi phục từ snapshot gần nhất rồi phát lại phần log còn thiếu, thay vì đồng bộ lại từ đầu. Đây là điều kiện tiên quyết để cluster sống sót qua sự cố mất điện đồng thời và để chạy thực tế trong môi trường sản xuất.

5.3.3 Đường ống hoá việc đóng block (pipelining)

Gỡ bỏ ràng buộc “một block in-flight” bằng cách cho phép leader đề xuất **nhiều block gối đầu** trước khi block trước đó được commit, tương tự cơ chế cửa sổ trượt. Hướng này trực tiếp nhắm vào cận trên thông lượng đã nhận diện ở Mục ?? và được kỳ vọng thu hẹp khoảng cách giữa trần đóng block đo trên một node và thông lượng thực tế nhiều node, đổi lại độ phức tạp trong quản lý trật tự commit và xử lý khi leader đổi.

5.3.4 Tối ưu đường nóng của Core

Vì thực nghiệm cho thấy cổ chai end-to-end nằm ở Core chứ không ở ordering (Mục ??), hướng tăng thông lượng toàn hệ thống hiệu quả nhất là tối ưu đường

Core: (i) **song song hoá xác minh chữ ký** endorsement và gộp lô kiểm tra; (ii) xem xét **chữ ký tổng hợp** (ví dụ BLS) để giảm số lần xác minh trên mỗi block; (iii) mở rộng pool sandbox WASM và cache module theo tải; và (iv) đo lại toàn pipeline sau tối ưu để xác định cổ chai kế tiếp.

5.3.5 Đánh giá trên cụm nhiều node và mạng thật

Lập lại bộ thực nghiệm ở Chương ?? trên cụm **nhiều node phân tán** qua mạng thật, với trạng thái ghi xuống đĩa và giao dịch thực thi WASM đầy đủ. Mục tiêu là lượng hoá phần chi phí mà các giả định hiện tại đã bỏ qua — RTT đồng thuận, độ trễ mạng, chi phí fsync — và kiểm chứng giả thuyết rằng thứ hạng tương đối giữa các cấu hình (đỉnh ở block=250–500, vùng trũng ở block=1.000) vẫn được giữ nguyên vì chúng bắt nguồn từ cơ chế gom lô độc lập với số node.

5.3.6 Cấu hình lại thành viên lúc chạy

Hỗ trợ thêm/bớt node và điều chỉnh độ ưu tiên **trong lúc cluster đang chạy** mà không cần khởi động lại, dựa trên cơ chế thay đổi cấu hình an toàn của Raft (joint consensus). Hướng này cần bảo toàn nguyên tắc quorum tính trên tổng số thành viên (Mục ??) trong suốt quá trình chuyển đổi để tránh split-brain.

5.3.7 Mở rộng theo chiều ngang: đa kênh và phân mảnh

Theo mô hình *channel* của Hyperledger Fabric, có thể cho phép nhiều cluster ordering độc lập phục vụ các kênh khác nhau, mỗi kênh một sổ cái riêng. Đây là con đường mở rộng thông lượng theo chiều ngang khi một cluster đơn đã đạt trần, đồng thời cung cấp cách ly dữ liệu giữa các nhóm tổ chức.

5.3.8 Hoàn thiện hệ sinh thái hợp đồng thông minh

Bổ sung một **SDK phát triển hợp đồng** đa ngôn ngữ (mọi ngôn ngữ biên dịch được sang WASM), cơ chế **vòng đời hợp đồng** (cài đặt, nâng cấp, thu hồi có kiểm soát), và **đo/giới hạn tài nguyên** (gas metering) để chống hợp đồng độc hại hoặc vòng lặp vô hạn — tận dụng khả năng cô lập sẵn có của **wazero**.

5.3.9 Siết bảo mật và sẵn sàng vận hành

Xây dựng hạ tầng danh tính đầy đủ (MSP/CA) cho các tổ chức thành viên, bắt buộc TLS/xác thực hai chiều trên toàn bộ kênh `libp2p`, và bổ sung phân quyền truy cập ở mức tổ chức. Song song, hoàn thiện quan sát hệ thống (observability) và đóng gói triển khai (ví dụ Kubernetes) để đưa các công cụ orchestrator hiện nằm ngoài phạm vi (Mục 1.5) lên mức sẵn sàng sản xuất.